

Unidad 1 - Ingeniería de software en contexto

Introducción a la ingeniería de software. ¿Qué es?

Software

- El software es un conjunto de programas en conjunto con **toda** la documentación requerida para definir, desarrollar y mantener los programas ejecutables que se entregan al cliente (archivos de configuración, documentos p/ el usuario, documentos sobre la estructura del sistema), como así también las herramientas utilizadas para la construcción de este.
- Tenemos 3 tipos básicos de SW
 - System Software
 - Utilitarios
 - Software de aplicación
- Casi ningún producto de SW es igual a otro, ya que los requerimientos son diferentes en cada uno.
- El software no es predecible, no se gasta (no deja de tener vigencia) sino que se va adaptando a los cambios de los requerimientos.
- No está gobernado por las leyes de la física

Ingeniería de Software

Es una disciplina que se encarga de todos los aspectos de la producción del software, desde la primera etapa de especificación del sistema hasta el mantenimiento del sistema después de que se pone en operación.

Ingeniería de Software → Parmas [1987] definió a la ingeniería en software como “multi-person construction of multi-version software”

Esta ingeniería es importante ya que:

- Con mayor frecuencia se requiere producir de manera rápida y económica sistemas confiables.
- Resulta más barato a largo plazo utilizar métodos y técnicas de ingeniería de software para los sistemas de software.
- La mayoría de los costos consisten en cambiar el software después de que se pone en operación.

Además, la ingeniería de software se apoya en varias realidades, entre ellas:

- Se debe entender el problema y las necesidades del cliente antes de proponer una solución.
- El diseño es una actividad crucial en el desarrollo de un software.
- Realizar un correcto diseño del software, trae consigo dos claros beneficios, calidad de software y facilidad de mantenimiento.

La ingeniería de software nace tras las **crisis del software**, en la cual existía (y existe) un conjunto de dificultades o errores ocurridos en la planificación, estimación de los costos, productividad y calidad de un software, debido, principalmente, a la baja eficacia que presentan una gran cantidad de empresas al momento de desarrollarlo.

Estado actual y antecedentes. La crisis del software.

El término crisis del software hace referencia a un conjunto de hechos relativos al software, planteados en la conferencia de OTAN en 1968 por Friedrich Bauer, quien recalco la dificultad para generar software libre de defectos, fácilmente comprensibles y que sean verificables. Sin embargo, este término fue utilizado anteriormente por Dijkstra en El Humilde Programador.

Causas

- La evolución de la tecnología del hardware mediante los circuitos integrados permitió la posibilidad del desarrollo de sistemas más grandes y complejos. El salto en el hardware no fue acompañado por un salto en el desarrollo del software.

- En combinación con lo anterior, la demanda creciente de estos sistemas complejos, la subestimación de la complejidad que supone el desarrollo de software, los cambios a los que tienen que ser sometidos los productos para ser adaptados a las necesidades del cliente y la falta de una disciplina que intervenga en todos los aspectos de la producción del software fueron las causas de esta crisis.

En la actualidad, muchos de los sistemas desarrollados fracasan debido a fallas en el software, que principalmente son consecuencia de dos factores:

- Demandas crecientes: la demanda de sistemas más grandes y complejos aumenta. Los usuarios necesitan que se construyan y distribuyan de manera rápida. La problemática aquí es que los métodos tradicionales no permiten lograr esta “espontaneidad”, por lo que se deben desarrollar y poner a prueba nuevas técnicas y métodos de gestión y desarrollo.
- Bajas expectativas: muchas de las compañías actuales de desarrollo de software no utilizan métodos de ingeniería de software en su trabajo diario. Por lo tanto, su software con frecuencia es más costoso y menos confiable de lo que debiera. La consecuencia, las empresas se conforman con que el software solo “funcione”, sin tener en cuenta el valor agregado que el software le debe aportar al negocio del usuario.

Disciplinas que conforman la ICW

Disciplinas Técnicas

- Toma de requerimientos
- Análisis de requerimiento
- Diseño de software
- Implementación de software
- Prueba de software
- Despliegue de un software
- Documentación de software
- Capacitaciones a usuarios

Disciplinas de Gestión

- Planificación de Proyecto
- Monitoreo y Control de Proyectos

Disciplinas de Soporte

- Gestión de Configuración de Software
- Aseguramiento de Calidad
- Métricas

Ejemplos de grandes proyectos de software fallidos y exitosos.

Software Exitoso

1. GNU/Linux: Es considerado un éxito rotundo por haber revolucionado la informática. Para septiembre de 2009, 446 de los 500 superordenadores más potentes del mundo ya utilizaban este sistema operativo. Además, es la base de los teléfonos con Android.
2. Apache: Es el servidor web por excelencia, alcanzando una cuota de mercado del 54% en 2010. Su versatilidad y su naturaleza de software libre le permitieron convertirse en el estándar de facto para servir páginas web.

Software Fallido

1. Ubuntu Phone: Se describe como un "fracaso espectacular". Su interfaz resultó casi imposible de usar para los usuarios, carecía de aplicaciones propias y la calidad de las llamadas telefónicas era absurdamente mala.
2. Steam Machine: Fue una consola de juegos basada en Linux desarrollada por Valve para atraer a los jugadores de PC al living de sus casas. Sin embargo, fracasó de tal manera que ni siquiera existe una explicación oficial clara, simplemente desapareció de los sitios oficiales

SCM – Gestion de configuración de SW

Es una disciplina de soporte, transversal a todo el proyecto con aplicación en diferentes disciplinas, que surge para enfrentar la crisis de problemas de software, aplicando dirección y monitoreo (administrativo y técnico) a:

- Identificar características técnicas y funcionales de ítems de configuración;
- Documentar características técnicas y funcionales de ítems de configuración;
- Controlar los cambios de esas características identificadas y documentadas;
- Registrar y reportar estos cambios;
- Verificar correspondencia con los requerimientos (trazabilidad).

Su propósito es resolver problemas de diferente índole, a través del establecimiento y el mantenimiento de la integridad del producto de software a lo largo de todo su ciclo de vida. Algunos de esos problemas son:

- Pérdida de componentes;
- Pérdida de cambios (la versión del componente no es la última);
- Regresión de fallas;
- Doble mantenimiento;
- Superposición de cambios;
- Cambios no validados.

Integridad de producto → debe satisfacer las necesidades del usuario, fácil y rastreable durante su ciclo de vida, satisfacer criterios de performance y satisfacer expectativas de costos.

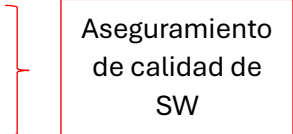
Mientras el producto existe hay actividad de gestión de configuración, que es responsabilidad de todo el equipo. Sin embargo, existen roles específicos como el rol del Gestor de configuración que tiene tareas adicionales como por ejemplo mantener la herramienta, marcar línea base, etc.

¿Cuál es el problema que se está tratando de solucionar?

El software es muy fácil de modificar, en el sentido de que uno puede entrar al repositorio y con un solo click eliminar meses de trabajo. Entonces lo que buscamos con la gestión de configuración del software es tratar de que estas cosas no pasen, buscando que el producto de software sea íntegro y si fuera el caso de que se van a realizar cambios, podamos tener un control de qué cosas cambiaron en un determinado momento.

Es por esta razón que se asocia el concepto de software con el concepto de versión, ya que cada ítem de configuración debe tener su versión. Y todas las personas se enteren de los cambios.

Disciplinas de soporte del SW

- Control de calidad de proceso
 - Control de calidad del producto
 - Prueba de SW
- 
- Aseguramiento
de calidad de
SW

¿Porque debemos gestionar la configuración?

- Para establecer y mantener la integridad de los productos de SW a lo largo de su ciclo de vida
- Esto involucra para la configuración:
 - Identificarla en un momento dado
 - Controlar los cambios
 - Mantener integridad y origen

Conceptos claves para la gestión de configuración de SW

- Ítem de configuración
 - Son aquellos artefactos que forman parte del producto o proyecto, y pueden ser almacenados en un repositorio, sin importar su extensión/tipo. Pueden sufrir cambios, y se desea conocer su estado y evolución a lo largo del ciclo de vida (ya sea del producto o proyecto, dependiendo del artefacto).
 - Es cualquier aspecto asociado al producto de software (requerimientos, diseño, código, datos de prueba, documentos, etc.) que es necesario mantener.
 - Son todos aquellos elementos que componen toda la información producida como parte del proceso de ingeniería de software, como ser programas de computadora (código fuente y ejecutables), documentos que describen los programas (documentos técnicos o de usuario), datos (de programa o externos).
 - Los ítems, dependiendo de su naturaleza, pueden durar lo que dure el ciclo de vida del proyecto, producto o sprint.
- Versión
 - Instancia de un ítem de configuración que difiere de otras instancias de este ítem. Las versiones se identifican unívocamente. Controlar una versión refiere a la evolución de un único ítem de configuración.
 - La versión es un punto particular en el tiempo de ese ítem de configuración (es un estado).
 - Una versión se define, desde el punto de vista de la evolución, como la forma particular de un artefacto en un instante o contexto dado.
- Repositorio
 - Es un contenedor de ítems de configuración, se encarga de mantener la historia de cada ítem con sus atributos y relaciones, además es usado para hacer evaluaciones de impacto de los cambios propuestos. Puede ser una o varias bases de datos. Tiene una estructura para mantener el orden y la integridad. El que tenga una estructura ayuda a la seguridad, los controles de acceso, las políticas de Backup y todo aquello que se aplica sobre un repositorio.
 - Repositorios centralizados: un servidor contiene todos los archivos con sus versiones. Los administradores tienen un mayor control sobre el repositorio. Tiene como desventaja qué si falla el servidor, todo lo positivo se cae.
 - Repositorio descentralizado: cada cliente tiene una copia exactamente igual del repositorio completo. Tiene como ventaja que se resuelve el problema de los servidores centralizados, debido a que si el servidor falla sólo es cuestión de “copiar y pegar”
- Release
 - Entrega de un sistema que se libera para su uso a los clientes u otros usuarios de la organización
- Ramas
 - Es un conjunto de ítems de configuración con sus correspondientes versiones, que permiten bifurcar el desarrollo de un software, por varios motivos: experimentación, resolución de errores en el desarrollo y desarrollar un mismo SW para diferentes plataformas.

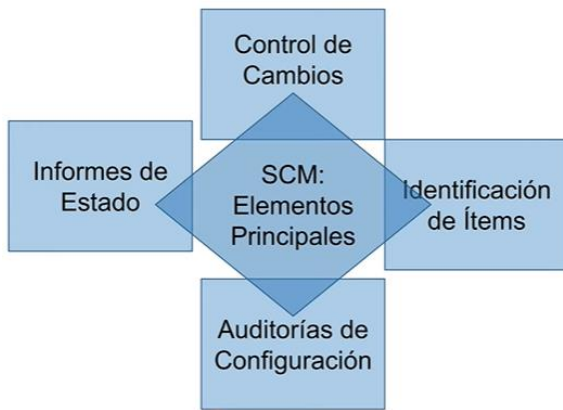
Linea Base

- Es un conjunto de ítems de configuración que han sido contruidos y revisados formalmente, de manera que pueden ser tomados como referencia para demostrar que se ha alcanzado cierto nivel de madurez en ellos, y que sirve como base para desarrollos posteriores. Esto es lo mismo que decir que es una configuración de software que ha sido formalmente revisada y aprobada, que sirve como base para desarrollos futuros.
- Este conjunto de ítems debe tener una referencia única, y esto se hace a través de “tags”.
- Se acuerdan parámetros para determinar qué se considera o qué debe cumplir los ítems de configuración para considerarse como línea base.
- Si se desea cambiar una línea base, existe un protocolo formal de control de cambios, dirigido por el Comité de Control de Cambios, que permite definir el procedimiento a seguir para manejar peticiones de cambios, y en caso de aceptarse, acordar nuevamente los parámetros y comunicar los cambios a todo el equipo, para que tomen la nueva línea base como modelo a seguir.
- ¿Para qué sirve una línea base? → Fundamentalmente es para tener un punto de referencia, pero también sirve para hacer Rollback o saber qué se pone en producción. Nos permite saber cuál era la última situación estable en un momento de tiempo y cómo se fue evolucionando. Permiten ir atrás en el tiempo y reproducir el entorno de desarrollo en un momento dado del proyecto.
- Tipos de líneas base
 - **Operacionales**: : contiene una versión de producto cuyo código es ejecutable, han pasado por un control de calidad definido previamente. La primera línea base operacional corresponde con la primera Release. Es la línea base de productos que han pasado por un control de calidad definido previamente.
 - **De especificación o documentación**: son las primeras línea base, dado que no cuentan con código. Podría contener el documento de especificación de requerimiento. Son de requerimientos, diseño.

Planificación de la gestión de configuración de SW

- Este plan se debe confeccionar al inicio de un proyecto, y existen diferentes estándares donde se expresa cómo planificar:
 - Reglas de nombrados de los ítems de configuración
 - Herramientas para utilizar en SCM
 - Roles e integrantes del comité de control de cambios
 - Procedimientos formales de cambios
 - Procesos de auditoria
 - Estructura del repo
 - SCM para SW externo
 - Como se hará el control de cambios
 - Registros que se deben mantener
 - Tipos de documentos

Actividades fundamentales de la administración de configuración de SW



Todas las actividades se realizan en ambas metodologías, menos las auditorías, ya que el concepto de ser auditado y controlado por alguien externo no es compatible con una metodología ágil pura.

Identificación de ítems de configuración

- implica una identificación unívoca para cada ítem, donde en el equipo se definirán políticas y reglas de nombrado y versionado para todos ellos. También, se debe definir la estructura del repositorio, y la ubicación de los ítems de configuración dentro de esa estructura.
- Esta identificación y documentación proveen un camino que une todas las etapas del ciclo de vida del software, lo que permite a los desarrolladores controlar y velar por la integridad del producto, como así también a los clientes evaluar esa integridad.
- También se debe tener en cuenta al momento de identificar los ítems, la duración de su integridad, ya que difieren en función del tiempo que es necesario mantenerlos.
- Tipos de ítems de configuración:
 - Ítems de producto
 - tienen el ciclo de vida más largo, y se mantienen mientras el producto exista. Por ejemplo, ERS, Arquitectura, código, manual de usuario
 - Ítems de proyecto
 - Plan de proyecto, cronograma
 - Ítems de iteración
 - Conocer el ciclo de vida de un ítem permite establecer la nomenclatura de este. Por ejemplo, plan de iteración, reportes de defectos
- Luego de identificar los ítems se debe armar la estructura del repositorio con nombres representativos y se vinculan los ítems con el lugar físico del repositorio donde se van a almacenar.

Control de cambios

- Una vez definida la línea base, no es posible cambiarla sin antes pasar por un proceso formal de control de cambios. Es decir que el control se hace sobre los ítems de configuración que pertenecen a la línea base, ya que todos los trabajadores del software tienen a dichos ítems como referencia.
- Comité de control de cambios
- Este proceso es llevado a cabo por el comité de control de cambios, el cual se reúne para autorizar la creación y cambios sobre la línea base. Forman parte de dicho comité los interesados en evaluar y enterarse del cambio, decidiendo si lo aceptan o no. El líder del proyecto, el analista, el arquitecto e incluso el cliente pueden constituirlo. Realmente la integración del comité depende de la propuesta del cambio.
- Si el comité fijó una línea base luego de la revisión de los ítems que la conforman y llegada a la conclusión de que los mismos se encuentran en estado estable, en caso de recibir una propuesta de cambio, los pasos son:
 - 1) Se recibe una propuesta de cambio sobre una línea base determinada, no sobre un ítem.
 - 2) El comité de control de cambios realiza un análisis de impacto del cambio para evaluar el esfuerzo técnico, el impacto en la gestión de los recursos, los efectos secundarios y el impacto global sobre la funcionalidad y la arquitectura del producto.

- 3) En caso de que se autorice la propuesta de cambio, se genera una orden de cambio que define lo que se va a realizar, las restricciones a tener en cuenta y los criterios para revisar y auditar.
- 4) Luego de realizado el cambio, el comité vuelve a intervenir para aprobar la modificación de la línea base y marcarla como línea base nuevamente, es decir que realiza una revisión de partes.
- 5) Notifica a los interesados los cambios realizados sobre la línea base.

Auditorías de configuración

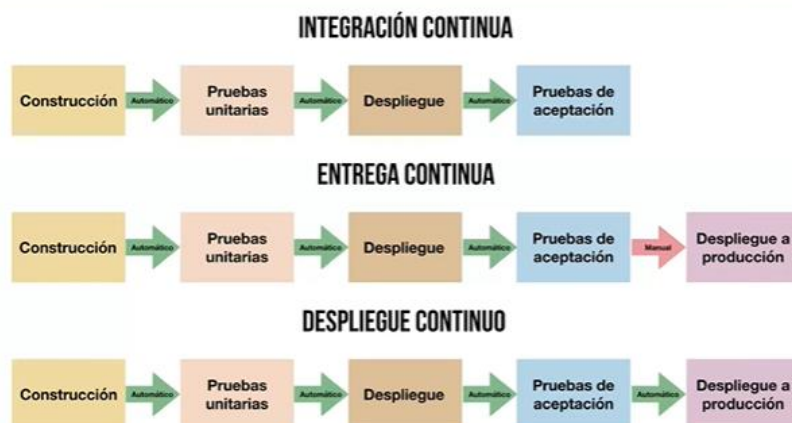
- Son controles autorizados a realizar por el equipo de desarrollo en un momento determinado, donde un auditor externo al equipo (independiente y objetivo) analiza las líneas base, las cuales permiten “congelar” y analizar en un momento determinado cuál es el estado del software, y si se están cumpliendo todas las pautas que plantea esta disciplina de SCM.
- En metodologías ágiles, esta es la única actividad de la disciplina SCM que no se soporta por la filosofía.
- La auditoría de configuración se hace mientras el producto se está construyendo y su objetivo es que el producto tenga calidad e integridad. Se entiende que lo más barato es prevenir.
- La auditoría de configuración complementa a la revisión técnica y consiste en revisar si se están realizando las tareas tales como se planificaron y especificaron en el plan de gestión de configuración. Las auditorías se realizan sobre una línea base. Es decir, la auditoría requiere la existencia de un plan de gestión de configuración y de la línea base a auditar.
- Establecer una correspondencia de relación entre ítems de configuración de diferentes niveles de abstracción en el proceso de desarrollo de software. Una traza puede ser constituida por el requerimiento, sus modelos de análisis y de diseño, la implementación en código y los casos de prueba para validar y verificar el requerimiento. La trazabilidad permite realizar un análisis del impacto sobre el sistema a partir de una solicitud de cambio en el requerimiento. Los modelos de calidad exigen la trazabilidad bidireccional. Desde el requerimiento hacia el componente de código y viceversa.
 - **Validación:** se encarga de asegurar que cada ítem de configuración resuelva el problema apropiado, es decir, lo que el cliente necesita.
 - **Verificación:** implica asegurar que un producto cumple con los objetivos definidos en la documentación de líneas base. Todas las funciones son llevadas a cabo con éxito y los test cases tengan status “ok” o bien consten como “problemas reportados” en la nota de reléase. Es decir, que se cumpla lo definido para cada ítem de configuración en la documentación de una línea base.
- Las auditorías permiten visualizar si se están satisfaciendo los requerimientos y si se ha cumplido la intención de la línea base anterior. Ante un análisis, el auditor puede detectar defectos y ajustar correcciones.
- Tipos de auditorías
 - **Auditoría física o de contingencia:** se realiza primero. Se validan los ítems de configuración de la línea base contra el plan de gestión de configuración, asegurando que la documentación que se entregará es consistente con el código desarrollado. Si esto no pasa, la auditoría funcional no se realiza.
 - Controla que el repositorio esté alojado en el lugar donde se dijo que iba a estar, que los IC respeten el esquema de nombrado, que estén guardados donde se dijo que iban a estar guardados.
 - Lo que se audita es una línea base.
 - Hace verificación.
 - **Auditoría funcional de configuración:** audita la trazabilidad de requerimientos, controlando que la funcionalidad y performance real de cada ítem de configuración sean consistentes con la especificación de requerimientos. El propósito final es validar que el código implementa únicamente y todos los requerimientos de la ERS.
 - Hace validación.

Ventajas	Desventajas
•aumenta la protección contra cambios innecesarios. •mejora de la visibilidad de estado del proyecto y sus componentes. •aumento de la auto responsabilidad •disminución de costos de retrabajo •disminución de tiempo de desarrollo •aumento de calidad •ayuda a la viabilidad y rastreabilidad del ciclo de vida del prod de SW.	•quejas •transición difícil. •no hay compromiso en todos los niveles. •no hay consciencia del problema

Informe de estado

- Provee un mecanismo para mantener un registro de cómo evoluciona el sistema, y dónde está ubicado el software, comparado con lo que está publicado en la línea base. Esto permite mantener a todo el equipo informado sobre la última línea base, y que se trabaje en base a su última versión, y no sobre una versión obsoleta.
- Estos reportes incluyen todos los cambios que han sido realizados a las líneas base durante el ciclo de vida del software. Normalmente esto consta de grandes unidades de datos, por lo que se utilizan herramientas automatizadas por una computadora.
- Su objetivo es asegurarse que la información de los cambios llegue a todos los involucrados.
- El reporte más conocido es el de inventario, el cual provee una copia del contenido del repositorio con la estructura de directorios.

Evolución de la gestión de configuración de SW



Continuous Integration

- Es una práctica de desarrollo que promueve que los desarrolladores adopten la costumbre de integrar su código a un repositorio compartido varias veces al día. Cada integración de código es luego verificada por una serie de pruebas automatizadas, permitiéndole al equipo detectar problemas de manera temprana. Ya que al integrar el código al repositorio de manera frecuente resulta más fácil detectar y corregir los errores.
- La integración continua es asegurar que el software pueda ser desplegado en cualquier momento, es decir, que el código compile y que sea de calidad.
- Cada desarrollador en su entorno de trabajo realiza pruebas unitarias y cuando terminó ese componente de código y lo probó y sabe que funciona, lo sube a un repositorio de integración.

Continuous Delivery

- Es una disciplina de desarrollo de software en la que el software se construye de tal manera que puede ser liberado en producción en cualquier momento. Suma la automatización de las pruebas de aceptación y entonces el producto ya está listo para desplegarlo a producción.
- No implica necesariamente que se libere cada vez que hay un cambio, sólo ocurre si el responsable de negocio, el Product Owner de turno, decide pasar a producción.
- La integración continua es prerequisite de la entrega continua. Con esto se refiere a que los artefactos producidos en el servidor de integración continua son puestos en producción con solo un click. Hay que asegurarse de tener un despliegue automatizado, para que cada puesta en producción no lleve demasiado tiempo, lo que hará que las puestas puedan llegar a hacerse más seguidas, generando que lo que se despliegue no sean demasiados cambios y el riesgo de que algo se rompa disminuya. El software debe estar siempre en un estado de “entregable”, es decir, el software buildea, el código se compila y los tests pasan.

Continuous Deployment

- Consiste en poner en el ambiente de producción del usuario final el producto. A diferencia de la entrega continua, en el despliegue continuo no existe la intervención humana para desplegar el producto en producción. Para esto, se utilizan pipelines que contienen una serie de pasos que deben ejecutarse en un orden determinado para que la instalación sea satisfactoria.
- El propósito de las estrategias es que sea transparente para el usuario que pusiste en producción una nueva versión del producto.
- Se recomienda hacer el despliegue a poco tiempo de haber trabajado con los cambios en el código, pues un error en producción un día después de haberlo hecho significa que todavía nos acordamos de lo que hicimos y será más fácil de resolver.
- Técnicas
 - Canary Deployment → es la práctica de realizar lanzamientos por etapas. Primero implementamos una actualización de software para una pequeña parte de los usuarios, para que puedan probarla y proporcionar comentarios. Una vez aceptado el cambio, la actualización se extiende al resto de usuarios.
 - Blue-Green Deployment → es un modelo de lanzamiento de aplicaciones que transfiere gradualmente el tráfico de usuarios de una versión anterior de una aplicación o microservicio a una nueva versión casi idéntica, las cuales se ejecutan en producción.

Gestión de configuración en ambientes ágiles

- En las metodologías ágiles la gestión de configuración cambia el enfoque, ya que la misma es de utilidad para los miembros del equipo de desarrollo y no viceversa.
- En orden con la caracterización de los equipos ágiles, decimos que la gestión de configuración posibilita el seguimiento y la coordinación del desarrollo en lugar de controlar a los desarrolladores.
- Los equipos ágiles son autoorganizados, por lo que las Auditorías de configuración no son una actividad propia de la gestión de configuración en los ambientes ágiles. Todos los procesos definidos establecidos en la gestión de configuración del enfoque tradicional son relativizados en agile.
- Podemos decir que va de la mano con el manifiesto ágil porque recordemos que se buscaba estar siempre preparados para el cambio y justamente, la gestión de configuración busca responder a los cambios y mantener la integridad del producto.
- La diferencia es que el manifiesto ágil se enfoca en los proyectos, mientras que la gestión de configuración de software se enfoca en el producto. Es decir, trasciende el proyecto.
- Mientras el manifiesto ágil apunta a cómo trabajar en el contexto del proyecto de desarrollo y la gestión de configuración es una práctica específica del producto que garantiza la calidad del producto.

Características

- 1) Dada la naturaleza de los requerimientos ágiles, la SCM responde a los cambios en lugar de evitarlos.
- 2) La automatización debe ser aplicada donde sea posible. Esto colabora con la entrega frecuente y rápida de software (integración continua).
- 3) Provee retroalimentación continua sobre calidad, estabilidad e integridad.
- 4) Las actividades de SCM se encuentran embebidas en las demás tareas para alcanzar el objetivo del sprint.
- 5) Es responsabilidad de todo el equipo, por lo tanto, requiere capacitación.
- 6) Adopción de las prácticas continuas.

Las auditorías es una actividad que por lo general no se realiza, todas las otras sí.

FILOSOFIA AGIL – MANIFIESTO AGIL

Manifiesto ágil

- Es un documento redactado en 2001 por expertos de la Ingeniería en Software, que promueve la filosofía del desarrollo ágil del software. Es un compromiso
- Esta basado en procesos empíricos, estos procesos trabajan con ciclos de vidas iterativos

Valores ágiles

- 1) Individuos e interacciones por sobre procesos y herramientas: es preferible un equipo de personas motivadas con un buen funcionamiento y conocimiento, donde la comunicación sea fluida, pero con herramientas pobres para su trabajo, que tener un equipo de individuos desmotivados o poco funcional con las mejores herramientas y procesos.
- 2) Software funcionando por sobre documentación extensiva: priorizar el software asegura que se tengan versiones estables, incrementales y mejoradas del software al finalizar cada iteración. Permitiendo que a medida que el software crezca en cuanto a funcionalidades, tras cada iteración puede ir mostrándose a los usuarios finales y obtener una retroalimentación.
Solo se documenta aquello que agregue valor al producto y esté centrado en el cliente (valor agregado). **Se debe generar la documentación cuándo sea necesario. No plantea “no generar documentación”.**
- 3) Colaboración con el cliente por sobre negociación contractual: esto permitirá no sólo tener una mejor noción de aquellos requerimientos de negocio que necesita el cliente, sino también que el equipo ágil se integre a todas las partes interesadas en el software, para que en conjunto puedan trabajar y lograr los mismos objetivos. Implica comprometer al cliente como parte importante del producto que se está construyendo. A veces las negociaciones contractuales imponen ciertas restricciones que impactan de manera negativa en las partes, lo cual se traduce en un mayor distanciamiento en la relación equipo de desarrollo-cliente.
- 4) Responder al cambio por sobre seguir un plan: esto se plantea debido a que es imposible que los usuarios sepan con detalle y certeza todas las funcionalidades que necesitan, y a su vez, con el paso del tiempo surgirán nuevas necesidades, las cuales en un primer momento no se habían detectado.
Los principales valores de la gestión ágil son la inspección y la adaptación, diferentes a los de la gestión de proyectos tradicional: planificación y control que evite desviaciones del plan.
Luego de cada iteración y haber desarrollado el mismo, se genera cierto conocimiento en el equipo, que afecta a la planificación. Debiendo incluir estos conocimientos en la planificación, apuntando siempre a entregar un producto con el mayor valor posible.

Principios del Manifiesto Ágil

- 1) Nuestra mayor prioridad es satisfacer al cliente
 - esto está ligado al valor 3 y 4, debido a que un equipo ágil desarrolla y entrega funcionalidades en el orden especificado por el Product Owner., que es quien determina la prioridad o importancia de cada funcionalidad o feature, de manera que en cada incremento o release del producto se retorne al cliente el mayor valor posible.
 - Está soportado por el hecho de trabajar con US, las cuales plantean las necesidades de los usuarios que tienen un valor significativo. De esta manera el equipo se asegura de responder dichas necesidades implementando las US.
- 2) Aceptar que los requisitos cambien
 - Incluso en etapas tardías del desarrollo esto puede suceder.
 - El equipo ágil está abierto al cambio, al no seguir un plan estructurado confeccionado en etapas tempranas, como lo plantea la metodología tradicional. Permitiendo que el equipo sea flexible y responda a las necesidades cambiantes del cliente
- 3) Entregar software funcional frecuentemente
 - Es imprescindible que al finalizar cada iteración (por lo general de 1 a 4 semanas) el equipo haya sido capaz de transformar uno o más requerimientos en código desarrollado completamente, testeado y potencialmente desplegable. Logrando que en cada iteración se le entregue al cliente el mayor valor posible
- 4) Técnicos y no técnicos trabajando juntos durante todo el proyecto
 - Gran importancia la participación del PO, en conjunto con el equipo de desarrollo. Este rol es ocupado por una persona del lado del cliente, siendo capaz de poder tomar decisiones sobre el producto. Representando el interés y necesidades comunes de todos los usuarios destinatarios del SW.
 - El PO es quien garantiza que en cada iteración los requerimientos serán implementados, siendo también el encargado de escribir las US.
- 5) Desarrollamos proyectos en torno a individuos motivados
 - la filosofía asume que aquellos individuos que participan en un proyecto de desarrollo del software deben estar lo suficiente motivados y comprometidos con los objetivos de este. Los participantes de un proyecto se deben ver entre sí como a un único equipo con objetivos comunes, donde las personas que asumen diferentes roles intentan colaborar entre sí para cumplir con esos objetivos, y no se limitan a cumplir con sus tareas y delegar los problemas en el otro.
- 6) El método mas eficiente de comunicar información es con conversaciones cara a cara
 - La forma más efectiva de comunicar es cara a cara (o lo más directa posible), ya que reduce malentendidos y mejora la rapidez en la toma de decisiones.
- 7) La mejor métrica del proceso es el software funcionando
 - El manifiesto ágil plantea que la mejor forma de demostrar el avance en un proyecto de software es a través de la entrega de software funcional y de valor para el cliente.
 - El verdadero avance se mide por lo que funciona, no por documentación o planes. Si el software no funciona, no hay progreso real.
- 8) Los procesos ágiles promueven el desarrollo sostenible
 - apunta a la estabilidad en equipos de trabajo a lo largo del tiempo, lo cual se traduce en una mejora continua de relaciones y formas de trabajo del equipo.
 - Además, mantener un equipo estable a lo largo del tiempo permite que el equipo alcance una madurez tal, en la cual es mucho más fácil realizar estimaciones sobre su participación en el proyecto.
- 9) La atención continua a la excelencia técnica y al buen diseño mejora la agilidad
 - manifiesta que la calidad del producto no es un aspecto negociable. Es de suma importancia que a medida que el software se desarrolla, se atiendan las necesidades primordiales del cliente y se responda a ellas.

10) La simplicidad es esencial

- apunta a que, como equipo de desarrollo, nos centremos en aquellas necesidades que realmente aporten un valor al cliente, y no perder el tiempo en detalles o aspectos que no se traducen en satisfacer requerimientos de los usuarios, ni hacer desarrollos de más.

11) Las mejores arquitecturas, diseños y requerimientos surgen de equipos autoorganizados

- este principio expresa que los equipos de desarrollo ágiles a diferencia de organizaciones jerárquicas tradicionales, donde la dirección/gerencia o los jefes son quienes imparten órdenes, y todos deben acatarlas, bajo esta filosofía los integrantes del equipo deben ser proactivos, aportando soluciones, conceptos, herramientas, etc. que sean útiles al equipo.
- Este principio está sustentado por el **empirismo**

12) A intervalos regulares, el equipo evalúa su desempeño y ajusta la manera de trabajar

- este principio se basa en los dos pilares de los procesos empíricos (inspección y adaptación), los cuales permiten a los equipos que cada determinado período de tiempo evalúe distintos aspectos concernientes a su desempeño, para adaptar aquellos aspectos que sean necesarios corregir, mejorar o incorporar al equipo, de manera que el comportamiento del equipo como un todo mejore.

Filosofía Ágil

- El objetivo primordial es entregar software de forma frecuente en un entorno cambiante.
- Estas metodologías se utilizan en entornos con gran variabilidad de requerimientos, involucrando al cliente en el proceso de desarrollo para conseguir una rápida retroalimentación.
- El desarrollo ágil adopta el ciclo de vida iterativo-incremental, donde el concepto del empirismo subyace al proceso, el software no se desarrolla como una sola unidad, sino como una serie de incrementos y cada uno de ellos incluye una nueva funcionalidad del sistema.
- Se reduce la cantidad de documentación con el uso de comunicaciones frecuentes e informales con el cliente en lugar de reuniones formales con documentos escritos.
- Los métodos ágiles son adaptables en lugar de predictivos y están orientados a la gente en lugar del proceso.
- La filosofía ágil se materializa en el manifiesto ágil. Éste está basado en que la forma de trabajo va a ser sustentada en los conceptos de los procesos empíricos, bajo 3 pilares fundamentales, la transparencia, la inspección y la adaptación.

Pilares del empirismo

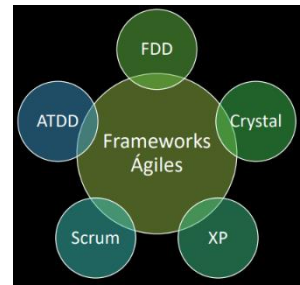
- Transparencia
 - nos permite crecer como equipos y transformar el conocimiento que cada miembro del equipo tiene implícito en conocimiento explícito, es decir, que pase a formar parte del conocimiento de todo el equipo.
 - Todo lo que desarrollemos y nos pase dentro del equipo, pasa a ser parte de todos.
- Inspección
 - La inspección puede realizarse para el producto, procesos, aspectos de personas, prácticas y mejoras continuas.
 - Se realiza por todo el equipo y permite detectar varianzas o problemas potencialmente indeseables, se debe inspeccionar el progreso hacia los objetivos acordados.
- Adaptación
 - Se trata de la mejora continua, es la capacidad de adaptarse en función de los resultados de la inspección.
 - Nos permite hacer ajustes en aspectos del proceso que se desvían fuera de los límites o si el producto es inaceptable.

Frameworks ágiles

- son distintos marcos de trabajo para implementar los principios y valores ágiles y generalmente son orientados a la gestión de proyectos y no al desarrollo.

Ser ágil no es ser indisciplinado

Hacer todo por pedacitos no es ser ágil



Requerimientos Ágiles

- Lo más difícil del software es definir que queremos construir, ya que la tecnología no es el impedimento, sino que los proyectos de SW fracasan debido a que no se cumplen con las expectativas del cliente (no es lo que el usuario necesitaba o quería).
- Como así también es posible que haya problemas de comunicación entre los clientes y el equipo, cuando ninguno de los 2 domina la comunicación el proyecto fracasa. Cuando el cliente la domina es probable que solicite funcionalidades y fechas de entregas que no puedan ser cumplidas por los desarrolladores o no se entienda lo que realmente quieren. Y cuando solo los desarrolladores la dominan la jerga técnica reemplaza al lenguaje del negocio y no escuchan al cliente.
- Lo óptimo es que haya una comunicación de ambas partes.

Fundamentación de los requerimientos ágiles

- El fondo de la cuestión está en los principios y valores explicitados en el manifiesto ágil, fundamentalmente la entrega frecuente y temprana de software valioso y la posibilidad de recibir cambios de requerimientos aun en etapas finales. Esto último implica una predisposición, una actitud del equipo frente a la posibilidad de cambios.
- Los requerimientos ágiles se basan en 3 pilares:
 - Usar el valor para construir el producto correcto
 - Lo único importante es dejar contento al cliente con software funcionando que le sirva, por lo que, el foco de este enfoque de gestión ágil apunta a construir valor de negocio.
 - Debemos enfocarnos en construir el producto correcto, logrando facilitar la vida a las personas que lo utilizan.
 - Determinar qué es lo suficiente
 - Los requerimientos deben ser encontrados de a poco, no se deben buscar todos desde un inicio y quedarnos solo con eso.
 - Se debe empezar con lo mínimo necesario para generar una retroalimentación y alimentar el empirismo, para que a partir de allí se avance de manera continua.
 - Usar historias y modelos para mostrar que construir
 - La parte de desarrollo de requerimientos está muy enfocada en construir junto con el cliente, respetando el principio de “técnicos y no técnicos trabajando juntos”.
 - Se busca trabajar con un referente que está del lado del negocio, con conocimiento del producto y que trabaje junto con el equipo. (PO en scrum y PC en XP)
- La etapa de capturar los requerimientos se transforma en una etapa de trabajo muy integrado entre el cliente y el equipo. Usando como medio de comunicación el cara a cara.

El costo tradicional BRUF – Big Requirements Up Front

A lo que se apunta con este concepto de BRUF es a notar que, en una primera instancia de un proyecto de software, desarrollar sólo unas pocas funcionalidades de gran valor e importancia para el cliente, puede cubrir gran parte de sus necesidades, a medida que se desarrolla el resto de los requerimientos.



Gestión Ágil de requerimientos

- Busca contrarrestar la situación planteada con anterioridad donde gran parte del software no se termina utilizando. El Product Backlog es una lista priorizada de características y requerimientos del software. Es el Product Owner quien se encarga de definir esta lista ya que es él quien tiene en claro cuáles son sus necesidades y requerimientos principales.
- El Product Owner se va a encargar de decidir qué requerimientos necesita primero según el valor que éstos tengan para el negocio.
- En contraposición con el enfoque tradicional, donde teníamos un cliente que no estaba dispuesto a comprometerse y dejaba la priorización de los requerimientos al equipo que hace el software, se terminaba construyendo un producto que no dejaba satisfecho al cliente.



Requerimientos Just in Time

- La gestión ágil de requerimientos sostiene los requerimientos justo a tiempo, asociados a los principios de eliminar el desperdicio y diferir compromisos, dentro de la filosofía Lean.
- Se empieza con una visión del producto y con algunos requerimientos identificados. A partir de esta base, se comienza a construir el producto y se incorpora al cliente para obtener retroalimentación. El Product Backlog nunca está al 100%.
- En el enfoque tradicional se aplica el "Up Front Specification", mediante el cual se especifica todos los requerimientos antes de comenzar el desarrollo. Esto implica que un error en la captura de requerimientos tiene un alto impacto.
- En cambio, los requerimientos en ambientes ágiles al ser consensuado continuamente con el cliente, el impacto que se tiene es menor. El producto no va a estar especificado al 100% desde el principio, sino que se van a ir encontrando requerimientos y describiendo los que hacen falta agregar conforme sea necesario "ni antes, ni después".

Fuente de requerimientos

- Los requerimientos ágiles, basados en el principio de que el mejor medio de comunicación es el cara a cara, promueven que los requerimientos del software se obtienen conversando cara a cara con el cliente.
- En los ambientes tradicionales, los requerimientos se encuentran especificados en un documento de especificación de requerimientos (ERS) y el cliente luego lo lee para validarlo.
- Los agilistas no escriben una ERS, pero lo compensan con la disponibilidad del cliente para resolver las dudas.

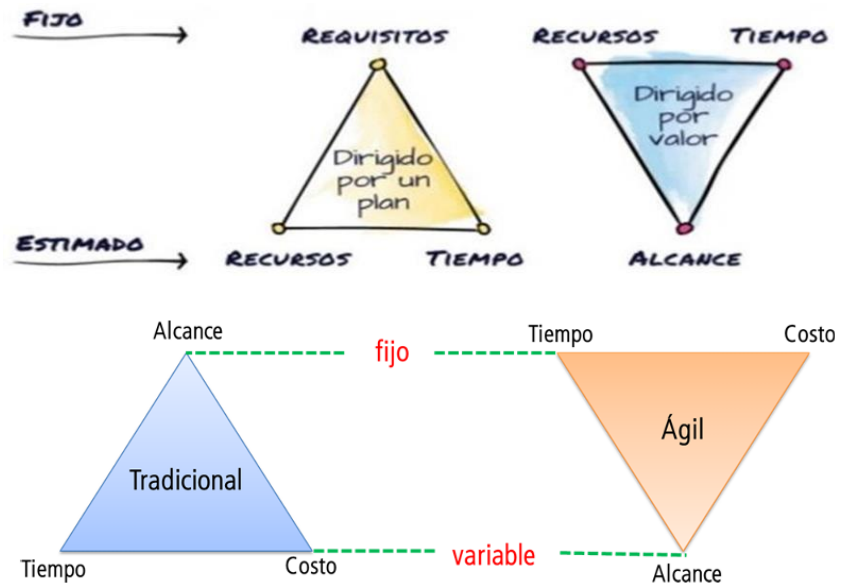
El cara-a-cara permite que fluya información vocal, subvocal, gestual con realimentación rápida.



La idea de generar valor de negocio es complicada, pero el SW es un medio de trabajo para el cliente. Debemos entregarle valor al cliente, mediante el SW.

Tradicional vs. Ágil

- En el enfoque ágil que el producto se define mediante lo que haga falta, en lugar de dejar fijo los requerimientos o el alcance y de ahí derivar en costos y tiempo. Sino que vamos a dejar fijo el recurso (equipo fijo) y el tiempo (sprint) asignado a trabajar. Estimando el alcance por iteración. Teniendo en cuenta cuanto SW le puedo entregar al cliente en tanto tiempo.
- En el enfoque tradicional el alcance se fija y en función de eso derivan los recursos y costos.



Tipos de requerimientos

- Los requerimientos de software no le interesan al cliente ya que es como el equipo transforma las necesidades en código ejecutable.
- Lo que si le interesa al cliente son los requerimientos de usuario y de negocio, ya que son parte del dominio del problema.
- Las US son requerimientos de usuario alineados a un requerimiento de negocio. Estas no sirven para especificar requerimientos de SW, sino que sirven para identificar los del usuario.
- El valor de negocio es el requerimiento de negocio, pero el de usuario es el medio por el cual vamos a lograr dicho valor.

¿Qué debemos asumir?

- Que los cambios son constantes
- Stakeholders: no son todos los que están, son todos los involucrados. El PO es el representante de todos ellos.
- Siempre se cumple lo de que “el usuario dice lo que quiere cuando recibe lo que pidió”
- No hay técnicas ni herramientas que sirvan para todos los casos.
- Lo importante no es entregar una salida o un requerimiento, sino que lo importante es entregar un resultado, una solución de valor.

Principios ágiles relacionados a los requerimientos ágiles



User Stories

- Son una técnica para trabajar requerimientos de usuario en ambientes ágiles. La US contiene una descripción corta de una funcionalidad que se espera del producto valuada por un usuario del sistema.
- Es un requerimiento a nivel de usuario, no al nivel de sistema, por lo que se encuentra a un nivel de abstracción más elevado. Es un ítem de planificación que no acompaña al producto durante todo el ciclo de vida. Para eso está el manual de usuario. Las US son escritas por el Product Owner.
- Las User Stories se consideran verticales dentro del producto, ya que pueden incluir aspectos desde diseños de interfaces hasta diseños de tablas en la base de datos.
- Cumplen diferentes funciones en el desarrollo de un SW:
 - Representan una necesidad de usuario de forma no detallada.
 - Conforman una descripción del producto
 - Son utilizadas como ítems de planificación
 - Se utilizan como recordatorios de futuras conversaciones acerca del producto a construir
 - Junto con el código ejecutable permiten documentar el producto.
- La definición de los requerimientos es lo que más afecta la calidad del sistema final y es lo más complejo de definir con detalle debido a la incertidumbre y cambio constante que sufren, que se da por el problema de comunicación.
- La elicitación tradicional no da buenos resultados, ya que todas las decisiones se toman al principio del proyecto cuando menos información se tiene.

Partes de una User Story

- 1) Tarjeta: es una descripción de la historia que se utiliza para planificar y como recordatorio.

<<Frase Verbal>>
Como <<nombre del rol>> yo puedo <<actividad>> de forma tal que <<valor de negocio>>

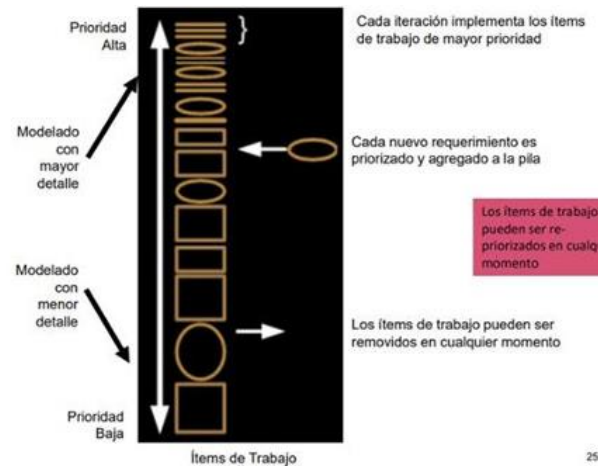
- - Nombre del rol: es quien está realizando la acción o quien recibe el valor de la actividad
 - Actividad: es la acción que realizara el sistema
 - Valor de negocio: es la mas importante y representa el porqué es necesaria la actividad.
 - Frase verbal: no es obligatoria, es una forma corta de referenciar la US. Representa la funcionalidad que se expresa en la User Story.
 - Criterios de aceptación: son diferentes criterios sobre la actividad que se necesita implementar, que permiten definir los límites de la U.S.
 - Pruebas de aceptación: acompañan a la tarjeta en la parte trasera, y es donde se capturan todos los detalles (que se manifiestan en la conversación) de la User, permitiendo determinar cuándo una historia está completa.
- 2) Conversación: es la parte mas importante de la US, explicita la comunicación entre el PO y el equipo de desarrollo, permitiendo compensar la falta de especificación y detalle.
- 3) Confirmación: son las pruebas de usuarios identificadas como necesarias para que el PO acepte la característica de SW construida para cada US.

Ventajas	Desventajas
•Prioriza la comunicación verbal. •Son comprensibles tanto para el cliente/usuario como para el equipo de desarrollo. •tienen el tamaño justo para permitir planificar. •Permiten el desarrollo iterativo. •Permiten desestimar los detalles, hasta que se logre una mejor comprensión sobre la necesidad concreta de los usuarios.	•En proyectos grandes es difícil entender la relación entre US. •No son apropiadas para equipos demasiados grandes.

Product Backlog y las User Stories

- Es una pila de PBIs (Product Backlog Ítems) con sus correspondientes estimaciones de tamaño, en un formato determinado. En la parte más alta de la pila, se ubican las U.S. de mayor prioridad y las de menor prioridad al fondo. En cualquier momento se puede cambiar esa priorización, según cómo lo indique el Product Owner.
- Para quitar una U.S. del Product Backlog, se planifica una iteración y se decide, según la velocidad de trabajo del equipo [Story Points / iteración], cuántas U.S. de la pila (las de mayor prioridad que están arriba) se quitarán para implementarlas en la próxima iteración.

El Product Owner Prioriza las historias en el Product Backlog



Las user stories son porciones verticales, es decir, que para poder agregar un poco de valor para el cliente debemos tener un poco de interfaz, un poco de lógica de negocio y un poco de BD.

Es decir, debemos diseñar el pedacito de cada cosa que nos hace falta para que una determinada característica funcione.

Story 1	Story 2
GUI	
Business Logic	
Database	

Tamaño de una US

- El tamaño de las U.S. se mide en Story Points.
- Hay 3 aspectos que determinan el tamaño de una U.S.:
 - **Complejidad:** hace referencia a la dificultad inherente a la funcionalidad, que no permiten dividirla en US mas pequeñas.
 - **Esfuerzo:** es el tiempo en horas lineales de trabajo que requiera la implementacion.
 - **Incertidumbre:**
 - **Técnica:** es cuando se tiene dudas en el dominio de la solución, por ejemplo, el uso de cierta tecnología no es muy conocida por el equipo.
 - **De negocio:** es aquella respecto a como el usuario interactuara con el sistema, es causada por falta de conocimiento del dominio del negocio.

Story Points

- Son unidades de medidas relativas que miden el tamaño de un producto, no de una medida basada en el tiempo. Ya que estimar basandonos en el tiempo conduce a errores.

Modelado de roles

- Se utilizan las tarjetas de rol de usuario, aquí se anotan las características en general que va a tener el usuario.
- El tema crítico de la funcionalidad está relacionado con quien va a utilizar dicha funcionalidad. Por esta razón existe una tendencia por ocuparse de quien va a usar el sistema, especificando los roles de usuario donde definimos las características del usuario respecto al producto y en relación con su perfil genérico.
- Técnicas adicionales
 - **Personas:** esta técnica busca ser más específica acerca de la descripción de la persona concreta que va a utilizar el sistema.
 - **Personajes extremos:** que personas en términos extremos podrían llegar a utilizar el software.
 - **Proxies (Usuarios Representantes):** es un representante de todos los usuarios. Debe tener capacidad de tomar decisiones desde el negocio.

Criterios de aceptación de US

- Define límites de la US y son las bases con las cuales se realizan las pruebas de usuario
- Ayuda a que los PO respondan lo que necesitan para que la US provea valor
- Ayudan a que el equipo tenga una visión compartida de la US
- Ayudan a desarrolladores y tester a derivar las pruebas. Y también a cuando parar de agregar funcionalidad en una US
- Se escriben a nivel de usuario

Pruebas de aceptación o de usuario

- Son declaraciones de intención de que hay que probar. El conocimiento del negocio está inmerso en las pruebas de aceptación. Se prueban escenarios exitosos y escenarios que fallen.
- El Product Owner acepta la User como implementada si todas las pruebas de aceptación se cumplen.

DoR (Definition of Ready) – Definición de Listo

- Es una medida de calidad que determina si la User Story está en condiciones de entrar a una iteración de desarrollo. Para que la US pueda ser implementada, mínimamente debe satisfacer el INVEST Model.

Invest Model

- Independiente: la User es calendarizable e implementable en cualquier orden.
- Negociable: debe estar escrita en términos de que necesita el usuario no como lo vamos a implementar.
- Valuable: las U.S. deben aportar valor de negocio a quienes estará destinado el producto, no para el equipo de desarrollo.
- Estimable: la US debe contener la cantidad de información suficiente para estimar el tamaño, la complejidad y el esfuerzo necesario para realizarlo.
- Small: la US debe ser lo suficientemente pequeña como para ser finalizada en una iteración.
- Testable: la US debe cumplir con las pruebas de aceptación para poder probar si la misma fue o no implementada

DoD (Definition of Done) – Definición de Hecho

- Determina si la US está terminada. Son los criterios que establecen cuándo una User Story ha sido implementada de forma correcta y completa, de forma que puede ser mostrada al Product Owner, para que éste decida si se pasa a producción o no.
- En el momento que una U.S. cumple con todo lo pactado en esta Definición de Hecho, se considera que esa US está lista para ser desplegada y aportar valor al cliente.

Niveles de abstracción

- 1) User Story
 - cumple con el criterio de Listo (DoR) para ingresar a una iteración.
- 2) Épica
 - Es una User muy grande. El tamaño es relativo a si entra en una iteración o no, ya que las User se empiezan y se terminan en la iteración. Si la User no entra en la iteración es considerado una épica. Tienen la misma sintaxis que la User.
- 3) Temáticas
 - Son un conjunto de User Stories agrupadas, debido a que conceptualmente están relacionadas. Se las agrupa para tratarlas como una entidad simple a la hora de estimar o planificar.

Spikes

- Son un tipo especial de US que se producen por la incertidumbre que la misma presenta, la cual imposibilita que pueda ser estimada y por lo tanto no cumple con la definición de listo, Una vez resuelta la incertidumbre, la Spike se convierte en una o más US.
- El equipo le tiene que dedicar tiempo para:
 - Familiarizarse con una nueva tecnología o dominio
 - Investigar y prototipar para ganar confianza frente a riesgos tecnológicos o funcionales.
- Se pueden utilizar para:
 - Inversión básica para familiarizar al equipo con una nueva tecnología o dominio.
 - Analizar un comportamiento de una historia compleja y poder así dividirla en piezas manejables.
 - Ganar confianza frente a riesgos tecnológicos, investigando o prototipando para ganar confianza.
 - Frente a riesgos funcionales, donde no está claro como el sistema debe resolverla interacción con el usuario para alcanzar el beneficio esperado.

Técnicas

- Utilizadas para investigar enfoques técnicos en el dominio de la solución.
 - Evaluar performance potencial
 - Decisión hacer o comprar
 - Evaluar la implementación de cierta tecnología.
- Cualquier situación en la que el equipo necesite una comprensión más fiable antes de comprometerse a una nueva funcionalidad en un tiempo fijo.

Funcionales

- Utilizadas cuando hay cierta incertidumbre respecto de cómo el usuario interactuará con el sistema.
- Usualmente son mejor evaluadas con prototipos para obtener realimentación de los usuarios o involucrados.

Lineamientos para spikes

- Son estimables, demostrables y aceptables
- La excepción, no la regla:
 - Toda historia tiene incertidumbre y riesgos.
 - El objetivo del equipo es aprender a aceptar y resolver cierta incertidumbre en cada iteración.
 - Los spikes deben dejarse para incógnitas más críticas y grandes.
 - Utilizar spikes como última opción.
- Implementar la spike en una iteración separada de las historias resultantes
 - Salvo que el spike sea pequeño y sencillo y sea probable encontrar una solución rápida en cuyo caso, spike e historia pueden incluirse en la misma iteración.

Estimaciones

- Estimar es el proceso de obtener una aproximación sobre una medida con el objetivo de predecir la completitud y gestionar los riesgos en el contexto de un proyecto. Se relaciona con los objetivos del negocio, compromisos y control.
- Las estimaciones no son planes ni compromisos, sino que sirven de base para planificar, pero el plan final no tiene por qué coincidir con lo estimado.
- Algunas consideraciones:
 - Por definición una estimación no es precisa.
 - Estimar no es planear y viceversa.
 - Las estimaciones son las bases de los planes, pero los planes no tienen que ser lo mismo que lo estimado.
 - A mayor diferencia entre lo estimado y lo planeado hay mayor riesgo
 - Las estimaciones no son compromisos

¿Para que estimamos?

- Hay dos razones principales
 - Para predecir completitud
 - Para administrar riesgos
- Antes de que el proyecto comience, el líder del proyecto y el equipo de software deben estimar el **trabajo** que habrá de realizarse, los **recursos** que se requerirán y el **tiempo** que transcurrirá desde el principio hasta el final.
- Las estimaciones requieren: experiencia, acceso a buena información histórica (métricas), valor para comprometerse con predicciones cuantitativas cuando toda la información que existe es cualitativa.

Errores en las estimaciones

- 1) Información imprecisa acerca del software a estimar o acerca de la capacidad para realizar el proyecto.
- 2) Demasiado caos en el proyecto
- 3) Imprecisión generada por el proceso de estimación
- 4) Una de las fuentes más común es omitir actividades necesarias para la estimación del proyecto tales como, requerimientos faltantes, licencias, reuniones, revisiones, etc.

Técnicas fundamentales de estimación

- Contar: es decir contar líneas de código, es lo más fácil porque es contar literalmente las líneas de código.

Métodos Utilizados para estimar

- Basados en la experiencia
 - Datos históricos
 - consiste en recolectar datos básicos de otros proyectos para ir generando una base de conocimientos que sean de utilidad para futuras estimaciones.
 - Esto no va acorde a la gestión ágil, que dice que la experiencia no se puede extrapolar a otros equipos.
 - Juicio experto
 - es uno de los más utilizados, el problema se presenta cuando se utiliza como única técnica de estimación el juicio experto puro. Existen dos técnicas para aplicarlo:
 - ❖ Juicio experto puro: un experto estudia las especificaciones y realiza una estimación. Esta estimación se basa fundamentalmente en la experiencia del experto. Dentro de la empresa hay un gurú estimador, mucha dependencia de la organización sobre él.

- ❖ Juicio Wideband Delphi: es similar al anterior pero grupal, un grupo de personas se reúne con el objetivo de converger a una estimación común, tanto en esfuerzo como en duración. Se toma de base para el poker planning

- Analogía
- Basados exclusivamente en los recursos
- Basados en la capacidad del cliente o exclusivamente en el mercado
- Basados en los componentes del producto o en el proceso de desarrollo
- Métodos algorítmicos

Estimaciones en ambientes Agiles

- La filosofía ágil plantea que quien estima, es quien debe hacer el trabajo, con lo cual es el equipo de desarrollo quien realiza las estimaciones, de forma colectiva (menos el Product Owner, no participa en estimaciones).
- Al ser de esta manera, la estimación se ajusta mejor, debido a que se generan debates entre los miembros del equipo, lo cual puede abrir puntos de vista no detectados por otros, haciendo que ese proceso de estimar se vuelva lo más preciso posible
- Las estimaciones se tienden a confundir con compromisos o planificación. Uno planifica basándose en la estimación, pero no son lo mismo.
- No es una instancia única, ya que a medida que se avance el proyecto, más información vamos a tener y más eficiente van a ser nuestras estimaciones.
- Las feature/stories se estiman con un tamaño relativo conocido como STORY POINTS
- Existe una serie de consideraciones importantes:
 - Son medidas relativas no absolutas.
 - No es una medida basada en el tiempo
 - Las personas no saben estimar en términos absolutos, somos buenos comparando.
 - Comparar es más rápido
 - Se obtiene una mejor dinámica grupal y pensamiento de equipo más que individual
 - Se emplea mejor el tiempo de análisis de las stories

¿Que se estima en ambientes agiles?

- Lo que se estima en ambientes ágiles, es solo el tamaño de las US (o Features) usando como unidad de medida los Story Points, y teniendo en cuenta 3 aspectos de estas: incertidumbre, complejidad y esfuerzo.
- Es importante tener en claro que no se estima tiempo, ya que la duración de los Sprints es fija

Tamaño

- Dado que el tamaño es lo que se compara, lo definimos como la medida de cuán compleja y grande es una US, además de cuánto trabajo es requerido para hacer o completar una Feature/US.
- También el tamaño es función de la incertidumbre de dicha US, definida la incertidumbre como la falta de información.
- El tamaño no es lo mismo que el esfuerzo ya que el esfuerzo indica las horas humanas lineales para el desarrollo de la US.
- Formas de estimar el tamaño:
 - Numeros del 1 al 10
 - Talles de remera: S, M, L se suele utilizar como escala de estimación para los ítems del product backlog.
 - Serie exponencial de base 2
 - Serie de Fibonacci

¿Para que se estima en agil?

- Para determinar cuantas US se puede comprometer el equipo a terminar en una iteracion
- Al terminar la iteracion contamos los SP correspondientes a las US aceptadas por el PO y obtenemos la velocidad del sprint, lo cual es una de las métricas fundamentales.

Tamaño vs Esfuerzo

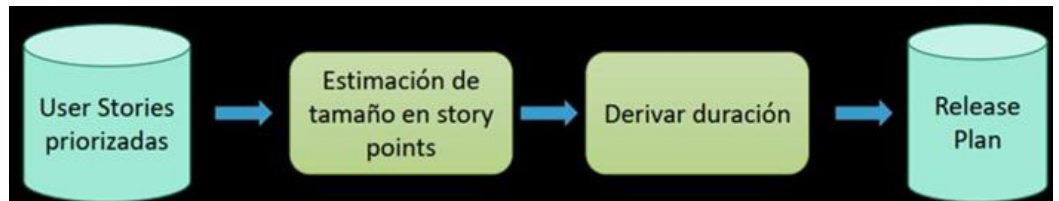
- Como el esfuerzo (horas de trabajo) es el factor más significativo del costo de un proyecto, tenemos una gran tendencia a confundir tamaño con esfuerzo y esfuerzo con calendario.
- La complejidad es independiente de quien haga el proyecto, y el esfuerzo depende específicamente de la persona que lo va a llevar a cabo, de su capacitación, conlleva horas lineales.
- Un trabajo puede requerir un esfuerzo de x horas, pero no se puede asegurar que para tal día va a estar terminado.

Velocidad

- Es una métrica del progreso de trabajo de un equipo. Se calcula (no se estima) sumando el número de Story Points asignados al estimar una US, que el equipo completa durante cada iteración, lo cual implica que la US se ha desarrollado y testeado por completo, y además el Product Owner la ha aceptado.
- Esta métrica permite retroalimentar el proceso de estimación, ya que un equipo maduro y con mucho trayecto, puede permitirse estimar cuántos puntos de historia puede desarrollar en la siguiente iteración a través del cálculo de velocidad.

Planificación de Release

- Se da al inicio del proyecto, donde se analiza y determina qué US, épicas o Temas serán desarrollados en una nueva entrega (Release) del producto. Este plan se va actualizando mientras el proyecto transcurre (usualmente al comienzo de cada iteración), de manera que siempre refleja las expectativas actuales sobre qué se incluye en la entrega.
- Una vez que se priorizan las US a implementar, se estima su tamaño con Story points, y teniendo en cuenta la velocidad de trabajo del equipo, se deriva la duración de la Release o del proyecto (la cantidad de sprints necesarios para obtener el release).



Poker Planning o Poker Estimation

- Es una técnica de estimación en ambientes ágiles. Reúne diferentes métodos como el juicio de experto, analogía y desagregación. En contraposición al método de juicio experto tradicional asume que las personas que desarrollan el producto deben ser las que estimen. Es decir, el equipo estima su propio trabajo, mezclando así el conocimiento de cada uno con la posibilidad de compartir con el resto del grupo. El PO participa en la planificación, pero no en las estimaciones.
- Escala
 - 1) Se utiliza la escala de Fibonacci para determinar los stories points
 - 1 o 1/2 son los valores mas bajos y son aplicados a funcionalidades pequeñas
 - 2 – 3 se aplican a funcionalidades pequeñas/medianas
 - 5 se aplica a funcionalidades medias
 - 8 se aplica a funcionalidades grandes en donde nos preguntamos si es posible dividir
 - 13 o mas se aplica a funcionalidades en las que necesariamente hay que dividir ya que no ingresa en el sprint.

- Prerrequisitos
 - Listas de features/US a estimar
 - Cada estimador tiene un mazo de cartas
- Pasos:
 - 1) Se determina la US canónica para comparar con las demás. Siendo esta la que tiene menor grado de incertidumbre, esfuerzo y complejidad.
 - Se lee la US a estimar
 - Cada estimador selecciona una carta
 - Se dan vueltas todas al mismo tiempo
 - Si todos seleccionaron el mismo valor, ese es el estimado. Si no se discute poniendo especial atención en los valores más bajos y más altos. Luego se repiten los pasos.
 - 2) Se toma la siguiente US
- Ventajas
 - Permite opiniones múltiples disciplinarias de todo el equipo, mejorando la estimación.
 - Al generar debates y discusiones se logra una mejor comprensión del dominio en el equipo
 - Reunir estimaciones individuales da mejores resultados que realizar estimación de forma grupal.
 - Es una actividad divertida para el equipo.

Diferencias con estimaciones en ambientes tradicionales

- En **metodologías ágiles** se estima el **tamaño o esfuerzo relativo** de las tareas, considerando factores como la **complejidad**, el **esfuerzo** y la **incertidumbre**. En **metodologías tradicionales**, en cambio, se estiman variables más concretas como el **tiempo**, el **costo**, los **recursos** y el **calendario del proyecto**.

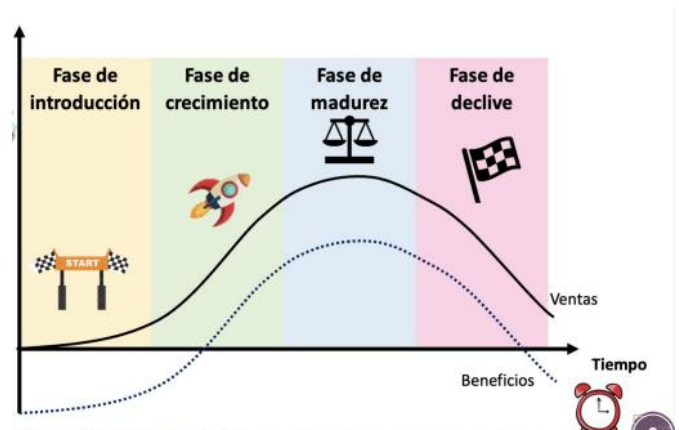
Gestión de Productos

¿Por qué creamos productos?

- Creamos para satisfacer necesidades específicas del cliente en base a ciertos requerimientos.
- Cuando creamos un producto debemos enfocarnos en aquello que es de valor para el cliente, sabiendo que el 7% de las características del software son siempre usadas. Esto implica, que la mayor parte del valor agregado está concentrado en ese 7%. Alrededor del 80% de las características del software que se desarrolla no se utilizan.
- Las organizaciones que usan el software se enfocan en la parte superficial. Usualmente no se fijan si el producto es funcional o confiable.
- También creamos para tener muchos usuarios logueados ya que es una métrica que nos permite saber que tan aceptado es el producto.
- Para obtener dinero
- Realizar una gran visión y cambiar el mundo

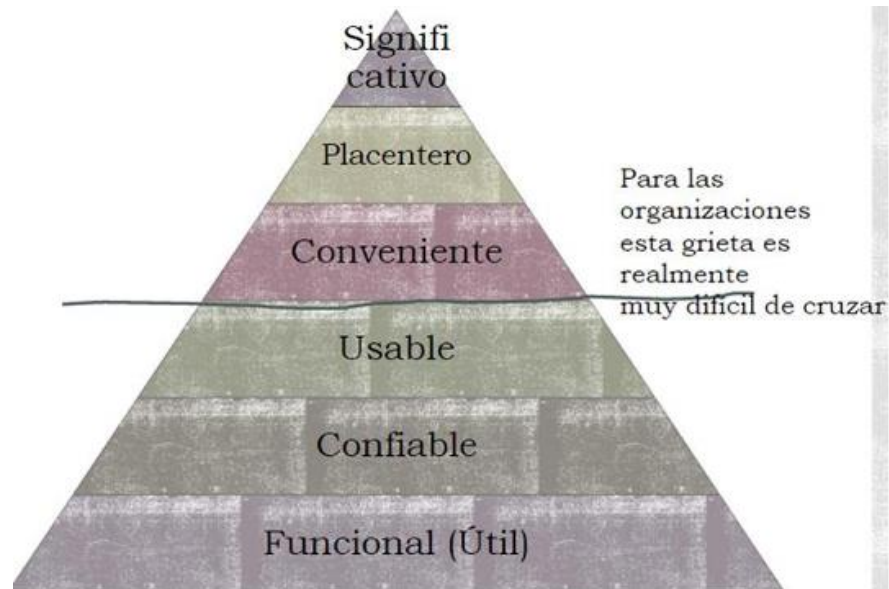
Ciclo de vida de un producto

- Tiene 4 fases: de producción, crecimiento, madurez y declive



Evolución de los productos de SW

- Cuando vamos a desarrollar un software la mayoría de las veces se empieza a construir un producto sobre la base de lo funcional, que haga lo que tiene que hacer, luego si se quiere avanzar más, el software debe ser confiable donde los usuarios no corran peligro usando el producto y los resultados que este producto me da yo pueda depender de ellos sin ningún problema.
- La usabilidad tiene que ver con aspectos de la disciplina UX/UI, donde el usuario podrá lograr sus objetivos productivamente usando este producto, estando a gusto con su uso. El usuario debe sentir que el producto le mejora la calidad de vida.
- Tiene que ser productivo, no es lo mismo que la utilidad.
- Existe una línea en la pirámide que separa la usabilidad de la conveniencia. La mayoría de los productos no se ubican por encima de esta línea, debido a que resulta complejo que un software sea conveniente para el desempeño de tareas de ciertas personas, y que estas no sean “esclavos” del software, limitándose a cumplir con las características de funcional, confiable y/o usable. Sino que también sea placentero y significativo para todo el mundo.



Planificación del producto ágil

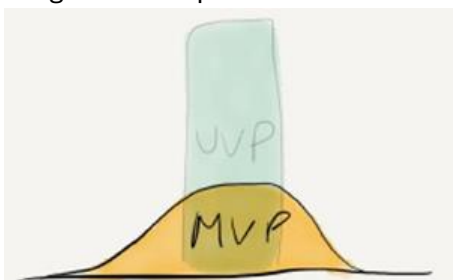
En las metodologías ágiles, se tiene una visión del producto materializado en el Product backlog, que contiene todas las características de la funcionalidad que requiere el producto que han sido identificadas hasta el momento. Esta visión se descompone en releases, las cuales son las entregas frecuentes que se hará al cliente. Cada release se construye a partir de sucesivas iteraciones.

Explicación de MVP, MVE, MME, con el dinosaurio de Lean/Agile

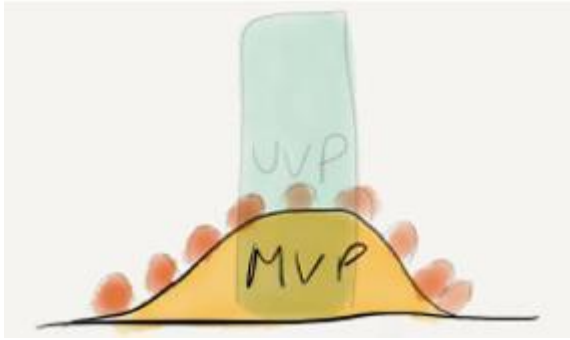
- 1) Comprender que un producto nuevo tiene una hipótesis de valor único, el producto será único (UVP).



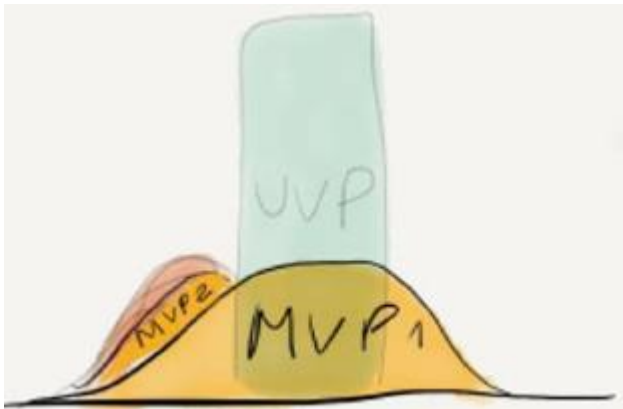
- 2) Como siguiente paso se crea un producto mínimo viable (MVP) para probar la hipótesis. Esto se centra en la propuesta de valor única, pero normalmente proporciona un poco de “tablestakes” solo para asegurarse de que sea viable como producto.



- 3) Tu MVP también es una hipótesis. Podría ser lo suficientemente bueno para encontrar un mercado o no. Se muestra el caso en el que cada cliente potencial con el que interactúas te dice "Esto es genial, pero para poder usarlo necesito X" y X es diferente para cada cliente / usuario. Esto muestra que aún no se encuentra un mercado para el producto.



- 4) Si por el contrario ves cada vez más respuestas apuntando al MISMO X entonces tiene sentido revisar la hipótesis de Cliente/Problema/Solución. Básicamente, estás ejecutando un pivot. Está construyendo MVP2 centrado en la nueva hipótesis basada en el aprendizaje reciente de Desarrollo de clientes generado por el anterior MVP.



- 5) Supongamos que MVP2 es exitoso y está viendo una atracción real de los primeros usuarios. Si desea aumentar el crecimiento y busca una penetración más profunda de sus primeros usuarios además de atraer nuevos clientes, algunos de ellos más allá de la multitud de usuarios pioneros. En función del feedback recopilado y la investigación de su gestión de productos, tiene un par de áreas que potencialmente pueden traer este crecimiento. Algunos de ellos por cierto amplían su propuesta de valor única y algunos hacen que su producto actual sea más robusto.



- 6) En el caso de áreas con una fuerte indicación de valor, puede directamente definir un MMF (Características mínimas comercializables). Para encontrar la pieza mínima que pueda empezar a traer crecimiento. El objetivo del MMF es aportar valor. Supone una alta certeza de que existe valor en esta área y que sabemos cuál debe ser el producto para proporcionar este valor. La razón para dividir una característica grande en MMF más pequeños es principalmente el tiempo de comercialización (Time to market) y la capacidad de aportar valor en muchas áreas. Una indicación de que está trabajando en MMF es que cuando al liberarse uno se siente cómodo trabajando en el próximo MMF en esa área.



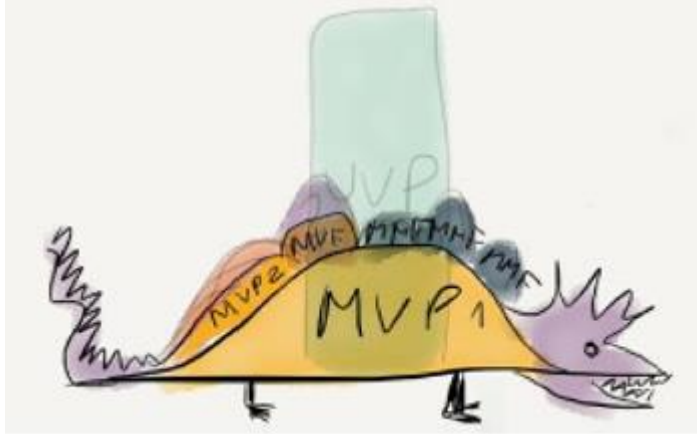
- 7) Si se desea esperar para ver si el primer MMF apuesta ... entonces está de vuelta en la tierra de la hipótesis. Ahora tu hipótesis se centra en una característica en lugar del producto. Tienes un área con alto potencial pero también alta incertidumbre.



- 8) La forma de afrontarlo es crear una función "pionera": MVF (Característica Mínima Viable). La característica mínima que aún puede ser viable para uso real y aprendizaje de los usuarios reales. Si el MVF resulta exitoso (hit gold), puede desarrollar más MMF en esa área para tomar ventaja (si eso tiene sentido). Si no es así, puede cambiar a otro enfoque hacia esa área de características, o en algún momento buscar una ruta de crecimiento alternativa. Esencialmente, el MVF es una versión mini del MVP.

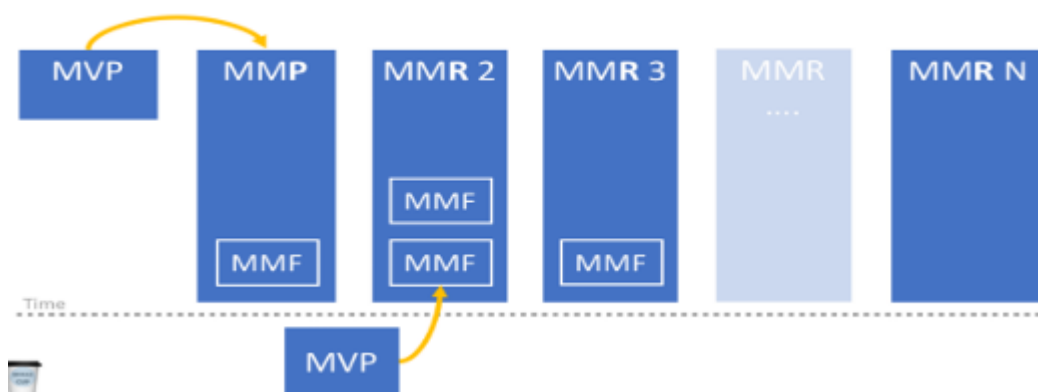


- 9) El producto se cultiva en mercados inciertos al intentar varios MVP. Cuando se logra ajustar el producto en el mercado de productos se combinan MMF y MVF según el nivel de incertidumbre del negocio / requisitos en las áreas en las que se está enfocando. Si bien los MVP / MMF / MVF son atómicos desde una perspectiva empresarial (no puede implementar y aprender de algo más pequeño) pueden ser bastante grandes desde la perspectiva de la implementación. El dinosaurio carpaccio se obtiene cortando cada una de esas piezas en pequeñas porciones destinadas a reducir el riesgo de ejecución / tecnología (normalmente se denominan User Stories). Esas porciones más pequeñas pueden tener un valor comercial tangible o no. Para otros puede ser importante proporcionar una temprana retroalimentación de decisiones a lo largo del camino.



Productos mínimos para la gestión de productos

- Producto Mínimo Viable (MVP). Tanto esta como el MVF responden a hipótesis.
- Característica Mínima Viable (MVF), es una versión mini del MVP.
- Característica Mínimas del Release (MRF). Son las características mínimas debe tener un producto para salir al mercado. Es una versión del producto que se publica y que el cliente puede usar.
- Característica Mínima Comerciable (MMF).



Empezamos todos esto, mediante las US, ver cuales forman parte del MVP para salir a validar.

Tener el MVP es tener una hipótesis a validar, por eso debemos tenerlo lo antes posible.

Busco validar si es el producto correcto o no.

Muchas veces lo confundimos con MMF porque queremos salir a vender un producto que no está en estado de madurez de producción, sino que está en uno de validación como lo es el MVP, podemos terminar en un fracaso.

Relación entre MVP, MMF, MMP, MMR

MVP	•Version de un nuevo producto creado con el menor esfuerzo posible. •Está dirigido a un subconjunto de clientes potenciales. • Utilizado para obtener aprendizaje validado. •Más cercano a los prototipos que a una version real funcionando de un producto. •Una premisa clave detrás de la idea de MVP es que usted produce un producto real que puede ofrecer a los clientes y observar su comportamiento real con el producto o servicio. •Ver lo que la gente realmente hace con respecto a un producto es mucho más confiable que preguntarle a la gente qué harían.
MMF	•Es la pieza más pequeña de funcionalidad que puede ser liberada (es una versión mini del MVP). •Tiene valor tanto para la organización como para los usuarios. •Es parte de un MMR o MMP.
MMP	•Primer release de un MMR dirigido a primeros usuarios (early adopters). •Focalizado en características clave que satisfarán a este grupo clave.
MMR	•Release de un producto que tiene el conjunto de características más pequeño posible. •El incremento más pequeño que ofrece un valor nuevo a los usuarios y satisface sus necesidades actuales. •MMP = MMR1.
MVF	•Es una versión mini del MVP. •Característica a pequeña escala que se puede construir e implementar rápidamente, utilizando recursos mínimos, para una población objetivo para probar la utilidad y adopción de la característica. •Un MVF debe proporcionar un valor claro a los usuarios y ser fácil de usar. •MVF requiere recursos mínimos, los estándares de calidad de la industria y la producción deben guiar el diseño y la confiabilidad. •El grupo de usuarios para un MVF, son los primeros en adoptar, los clientes leales que han compartido conocimientos anteriormente o los miembros de una junta asesora de clientes. Usuarios flexibles y tolerantes. •Los resultados le ayudarán a tomar decisiones estratégicas sobre productos

Valor vs Desperdicio

- La productividad de un Startup no puede medirse en términos de cuánto se construye cada día, por el contrario, se debe medir en términos de construir lo correcto cada día y lo correcto es aquello que agrega valor. Si no construimos valor, entonces estamos invirtiendo esfuerzo en desperdicio.

La fase de construir: MVP

- Ingresar lo más rápido posible con un Producto mínimo viable (MVP).
- Un MVP varía en complejidad desde pruebas de humo (smoke tests) extremadamente simples (poco más que un anuncio) hasta prototipos tempranos



Dilema de audacia cero: cuando no tenemos nada que perder, es cuando más nos animamos a arriesgar. En cambio, si tienes compromisos, clientes y demás ya no es tan fácil.

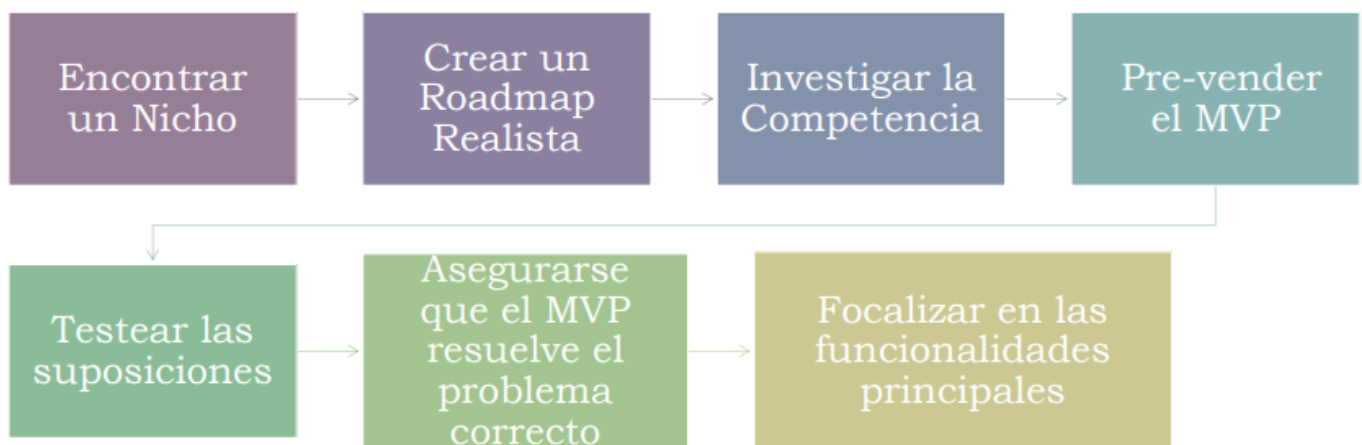
Supuestos saltos de Fe

- Los elementos más riesgosos del plan / concepto de una startup (es decir, las partes de las que todo depende) se denominan supuestos de salto de fe.
- La mayoría de las personas no conocen una determinada solución (o incluso un problema); pero una vez que experimentan la solución, ¡no pueden imaginar cómo vivirían sin ella!

Un MVP nos permite validar diferentes hipótesis:

- Hipótesis del valor: prueba si el producto realmente está entregando valor a los clientes después de que comienzan a usarlo (ver si el producto es realmente el producto correcto). Una métrica de prueba: tasa de retención (más de la mitad de los usuarios volvieron a utilizar el sistema todos los días, solo un 25% dejó de usarlo).
- Hipótesis de crecimiento: prueba cómo nuevos clientes descubrirán el Producto (ver si más personas desean usar el producto). Una métrica de prueba: tasa de referencia o Net Promoter Score (NPS) (Se corrió la voz acerca de lo que era, no fue siquiera necesario el uso de políticas de marketing o publicidad).

Preparar un MVP



Características de un MVP

- Diseño
 - Diseño adecuado
 - Consigue una UX que deleite
 - Logra satisfacer el aspecto visual y de interacción
- Usabilidad
 - Tiene suficiente valor para que la gente esté dispuesta a usarlo/comprarlo.
 - Resulta útil para su público objetivo
- Confiabilidad
 - Involucra a los early adopters puedan confiar en la solución plenamente
 - Incluso cuando tenga poco tiempo en el mercado
- Funcionalidad
 - Tiene las funciones necesarias para solucionar un problema específico.
 - Satisface las demandas y permite evaluar las funciones a implementar más adelante.

Matriz de priorización para el MVP

	Urgente	No urgente
Importante	1. Hacer Incluir en el MVP	2. Planear Desarrollar para un lanzamiento beta
No Importante	3. Delegar Considerar integraciones de terceras partes.	4. Eliminar Remover del Roadmap del producto

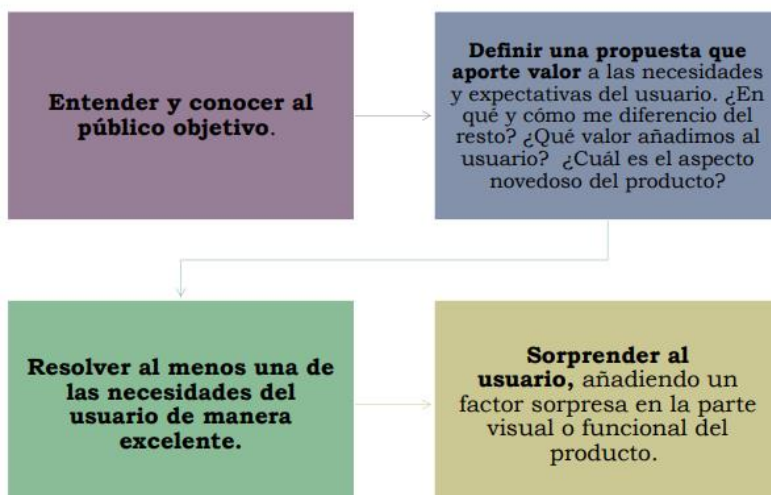
Minimal Lovable Product – MLP (Producto Mínimo Adorable)

- Es la versión más simple de un producto que genera una conexión emocional o deleite en los usuarios. Enfatiza la satisfacción del usuario y la diferenciación.
- fomenta la lealtad y la adopción temprana.
- Marca el nivel mínimo de diseño/desarrollo para que los clientes lo amen en lugar de simplemente tolerarlo.
- La idea es crear un producto viable mínimo que, además de proporcionar información al equipo de trabajo para el desarrollo futuro, apele a la emocionalidad del usuario para lograr lo más difícil: enamorarlos.

¿Por qué deberíamos crear un MLP?

- Reputación de Marca
 - Si somos una gran empresa cada producto que se lance se debe mostrar a los usuarios que nos importan las características que necesitan. De lo contrario esos productos pueden parecer un intento de lucro.
- Productos destacados
 - Las startups que intentan competir con competidores bien financiados necesitan crear un producto mínimo atractivo que destaque en el mercado. Los productos mínimos viables simplemente no son suficientes para revolucionar un mercado con soluciones consolidadas.
- Ciclo de retroalimentación del usuario
 - Al captar la atención de los usuarios desde el principio, podrás capitalizar su interés a medida que el producto mínimo atractivo se desarrolla. Esto te proporciona un flujo fiable de retroalimentación del usuario que puedes aprovechar para la mejora continua.

Claves para definir un MLP



Como construir un MLP

- Sentir la curiosidad: no debemos adivinar los que valoran los clientes, debemos preguntarles directamente.
- Conocer el mercado: debemos comprender el panorama en el que nos encontramos antes del desarrollo, luego debemos concentrarnos en satisfacer las necesidades de los clientes antes de expandirnos hacia otros.
- Definir el propósito: depurar lo que aprendemos de los clientes y el mercado para crear una visión de lo que se desea lograr.
- Centrarse en el conjunto: debemos recordar que el producto no es solo un conjunto de características, sino también todos los puntos de contacto con la empresa a lo largo de todo el recorrido del cliente.
- Construir para durar: debemos tomar decisiones arquitectónicas con seguridad en respuesta a las necesidades o problemas de los clientes, hay que confiar en el compromiso asumido con la solución que vamos a plantear.
- Dedicar esfuerzo: no debemos escatimar gastos, debemos invertir el esfuerzo y los recursos necesarios para brindar un buen servicio a los clientes desde el principio.
- Medir el cariño: debemos elegir una forma de medir el cariño que despierta el producto en los clientes.

Elementos claves de un MLP

Aspecto	Funcionalidad	Usabilidad	Deleite
Rol	Resolver el problema del usuario	Asegurar facilidad de uso	Evocar emociones positivas
Foco clave	Aspectos core	Interface Intuitiva	Interacciones Intuitivas
Beneficio Principal	Satisfacer necesidades básicas	Reducir fricción	Fomentar la lealtad
Ejemplo	App de streaming de música	App con navegación simple	App con playlists personalizadas

MVP – MMP – MLP

MVP vs. MLP

Producto Mínimo Viable (MVP). Se centra en lograr su función principal para que el concepto del producto pueda validarse de la manera más económica y rápida posible. Este suele ser el enfoque predilecto de las empresas que tienen prisa por lanzar un producto al mercado.

Producto Mínimo Adorable (MLP). Se centra en características que no solo son funcionales o útiles, sino que también son lo suficientemente impactantes como para deleitar a los usuarios. Esto requiere una investigación de usuarios sobre lo que los clientes consideran valioso y, por lo tanto, su lanzamiento llevará más tiempo que un MVP.

MMP vs. MLP

Producto Mínimo Comercializable (MMP). Es un producto listo para su lanzamiento al mercado. Ya es capaz de cubrir una amplia gama de funcionalidades, atendiendo a grandes bases de usuarios, e incluso podría ser una versión posterior de un MVP anterior (aunque probablemente aún necesite ajustes de UX/UI).

Producto Mínimo Adorable (MLP). Es un producto que prioriza la interacción del usuario y enfatiza la creación de una conexión emocional con los clientes que lo prueban. Esto contrasta marcadamente con los MMP, que se centran en un mayor atractivo para el mercado y la disponibilidad para el lanzamiento.

Componentes de Proyecto de Desarrollo de Software

Proceso de Software

Definición: Es un conjunto de actividades, métodos, prácticas y transformaciones que la gente usa para desarrollar o mantener SW y sus productos asociados.

- Conjunto estructurado de actividades para desarrollar un sistema de SW
- Estas varían dependiendo de la organización y el tipo de sistema a desarrollar
- Debe ser explícitamente modelado si va a ser administrado
- Las actividades del proceso de desarrollo son la especificación de requerimientos, el análisis, el diseño, la implementación y la prueba.
- El orden y la relación entre las diferentes actividades está determinada por la elección del ciclo de vida.

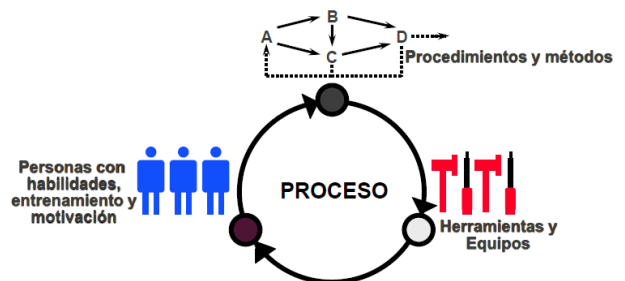


Tipos de procesos

Definido	Empírico
•Podemos repetir el mismo proceso una y otra vez, indefinidamente, y obtener los mismos resultados. •La administración y el control provienen de la predictibilidad del proceso definido Misma entrada → Un proceso definido → Misma salida Inspirados en líneas de producción	•Asume procesos complicados con variables cambiantes. •Cuando se repite el proceso se pueden llegar a obtener salidas diferentes. •La administración y el control es a través de inspecciones frecuentes y adaptaciones. •Son procesos que trabajan bien con procesos creativos y complejos. Asumir → Construir → Retroalimentar → Revisar → Adaptar

Se tienen tres factores determinantes del proceso:

- **Procedimientos y Métodos:** Deben **adaptarse al contexto** específico del proyecto y estar debidamente documentados para garantizar la transparencia y visibilidad de la gestión.
- **Personas:** Es el factor más crítico, ya que el desarrollo es una **actividad "humano-intensiva"**. El éxito depende del esfuerzo, la capacitación y la motivación de los individuos para resolver la complejidad conceptual del sistema.
- **Herramientas y Equipos:** Actúan como soporte para transformar entradas en salidas. Se busca **automatizar tareas rutinarias** (mediante herramientas CASE o IDEs) para que el personal pueda enfocarse en actividades que requieren su capacidad intelectual.



Ciclos de vida (Modelos de Proceso) y su influencia en la Administración de Proyectos de Software.

Ciclo de vida → una serie de pasos a través de los cuales un producto o un proyecto progresa en el tiempo.

Define:

- Elementos del proceso (actividades estructurales, productos del trabajo, tareas, etc.)
- El flujo del proceso (o flujo de trabajo), que especifica la relación y el orden de dichos elementos.

Además, establece los criterios que se utilizan para determinar cuándo se procede de una tarea a otra.

Los modelos de ciclos de vida especifican:

- Las fases del proceso (requerimientos, especificaciones, diseño, etc.)
- Orden en el cual se llevan a cabo.

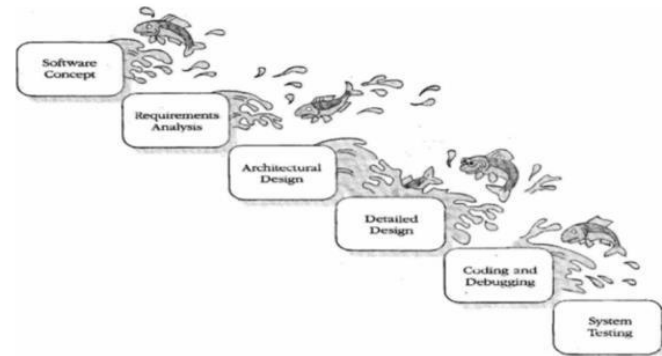
Tipos básicos de ciclos de vida

Modelo Secuencial

Este modelo dispone de las actividades de forma lineal, es decir, que el proyecto progresa a través de una secuencia ordenada de pasos (fases) y una actividad no puede iniciar sin que la precedente haya sido finalizada. El avance entre las fases se da tras una revisión al final de cada una de estas para determinar si el software está listo para avanzar.

Se suele utilizar en aquellos sistemas donde los requerimientos se comprenden bien y son exhaustivos ya que éstos se congelan al finalizar la etapa de toma de requerimientos.

Generalmente, este tipo de modelos está dirigido por documentos, es decir, el trabajo principal del producto es la documentación del software entre las distintas fases.



Modelo iterativo/incremental

Este modelo aplica sucesivas iteraciones en forma escalonada a medida que avanza el calendario de actividades.

Cada iteración produce un incremento de software funcional potencialmente entregable.

Usualmente los primeros incrementos incluyen las funciones básicas/críticas que más requiere el cliente.

Este enfoque vincula las actividades de especificación, desarrollo y validación.

El sistema se desarrolla como una serie de versiones (incrementos) y cada una añade funcionalidades a la versión anterior.

Este ciclo de vida se utiliza en metodologías ágiles.

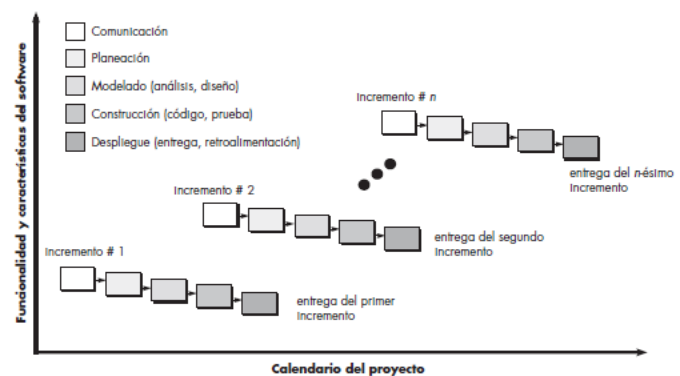
Modelo Recursivo

El modelo recursivo es utilizado para gestionar los riesgos del desarrollo de sistemas complejos a gran escala. Requiere de la intervención del cliente. El modelo en espiral es un modelo recursivo, que consta de 4 etapas recursivas:

1. Determinar objetivos.
2. Identificar y resolver los riesgos.
3. Desarrollar y probar.
4. Planificar la próxima iteración.

Para llevarlo adelante, se subdivide el proyecto en varios mini proyectos, intentando resolver en cada uno los riesgos más relevantes hasta que no quede ninguno.

Dicho en otras palabras, se inicia con algo en forma completa, como una subrutina que se llama a sí misma e inicia nuevamente. Se presenta un prototipo que va mejorando con cada vuelta. Lo positivo, es que se generan productos independientes de la implementación, que pueden ser reusables en sistemas de características similares.



Ciclo de vida	Ventajas	Desventajas
Secuencial	• El modelo en cascada puro ayuda a minimizar los gastos de planificación ya que se puede realizar por adelantado. No proporciona resultados tangibles del SW hasta el final del ciclo de vida, pero la documentación generada proporciona indicaciones de progreso a lo largo del ciclo de vida • Funciona bien para productos que tienen una definición clara y estable o cuando se trabaja con metodologías técnicas claras.	• Los proyectos raramente siguen un flujo lineal. • Exige que los requerimientos sean completamente explícitos desde un principio. • El cliente obtiene una versión funcional del producto en etapas muy avanzadas del proyecto. • Encontrar un defecto implica gran rediseño en la solución. • Dependencia entre los equipos de trabajo. • El desarrollo está dirigido por un plan, y cada etapa genera documentación para un monitoreo, la misma puede ser muy burocrática y excesiva.
Iterativo/Incremental	• Se reduce el costo de adaptar los requerimientos cambiantes del cliente y el retrabajo es menor, ya que en cada incremento se obtiene una retroalimentación, permitiendo la adaptación. • El cliente es parte del proceso de desarrollo. • Es posible que la entrega sea más rápida y la implementación de SW útil al cliente, aun si no se han incluido todas las funcionalidades. • Muy útil para sistemas con muchos cambios de requerimientos. • Las especificación, desarrollo y validación se entrelazan.	• Se invisibiliza el proceso. • Alto costo de documentar cada incremento. • La estructura del SW tiende a degradarse frente a una cantidad considerable de incrementos.
Recursoivo		• Puede ser mas costoso en tiempo y dinero, readaptarlos para reutilizarlos en diferentes proyectos. • La tecnología puede ser obsoleta. • Puede carecer de mantenimiento o documentación, dificulta mucho las tareas. • Se genera una versión útil del producto recién al final.

Relacion entre proceso, proyecto, persona, herramientas y producto

- La diferencia entre proceso, proyecto y producto es que el **proceso** es una plantilla (definición abstracta) que existe o se materializa a través del **proyecto** adaptándose para las necesidades de este. El resultado del proyecto es un **producto** de software.
- La diferencia mas importante es que el proyecto empieza y termina en un lapso, el producto sobrevive a la vida del proyecto.
- Las personas son claves para ejecutar las actividades del proyecto
- Las herramientas son para automatizar el proceso.

¿Qué es un proyecto?

- Un proyecto de software es un esfuerzo temporal que requiere del acuerdo de un conjunto de especialidades y recursos para la creación de un producto, servicio o resultado único. Es una unidad organizativa para adaptar el marco teórico del proceso, dependiendo de las personas y recursos con los que se cuente.
- Define el proceso y tareas que se van a realizar, el personal que se encargará de las actividades y los mecanismos que se implementarán para valorar riesgos, controlar el cambio y evaluar la calidad.

Características de un proyecto

- Deben ser únicos: El resultado de un proyecto será único e irrepetible para cualquier otro proyecto, por más parecido o igual sean sus elementos componentes. Todos los proyectos por más similares que sean tienen características propias que los hacen únicos.
- Orientados a objetivos: Los proyectos están dirigidos a obtener resultados, reflejándose a través de objetivos, que son quienes guían al proyecto. Estos objetivos deben ser:
 - No ambiguos: suficientemente claros para guiar el desarrollo del proyecto.
 - Medibles: es necesario medir el objetivo para determinar el avance sobre el proyecto.
 - Alcanzable: deben ser factibles de realizarse por el equipo
- Duración limitada de tiempo: Los proyectos son temporarios (tienen inicio y fin), cuando se alcanzan los objetivos el proyecto termina. Una línea de producción no es un proyecto.
- Tareas interrelacionadas basadas en esfuerzo y recursos: se definen tareas, dependencias, recursos y personas asignadas, permitiendo manejar la complejidad de los sistemas.

Relación entre producto de SW, proceso de SW y ciclo de vida

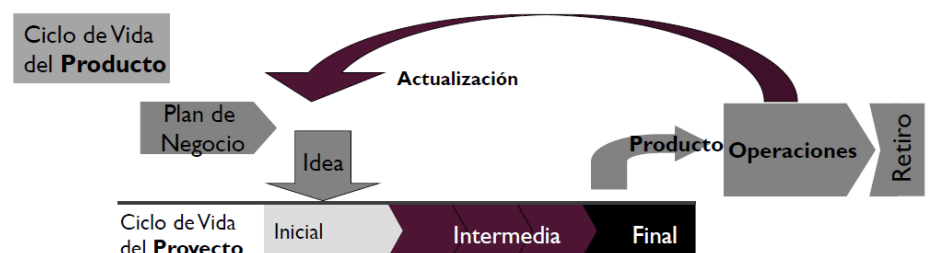
Para construir un producto se utiliza un proceso y ese proceso está dirigido por el ciclo de vida.

Relación entre el ciclo de vida del proyecto y del producto

No existe un impacto entre los ciclos de vida de cada uno, sino que el ciclo de vida del producto siempre es mayor que el ciclo de vida del proyecto, ya que el ciclo de vida del proyecto dura lo que dura el desarrollo del software. En cambio, el ciclo de vida del producto dura hasta que el software se deje de utilizar.

Es posible que un producto tenga varios proyectos en su ciclo de vida, debido a, constantes cambios y/o actualizaciones que se vayan realizando.

Por lo que, dentro del ciclo de vida del producto, se pueden desarrollar varios ciclos de vida de proyectos.



Administración de proyectos de software

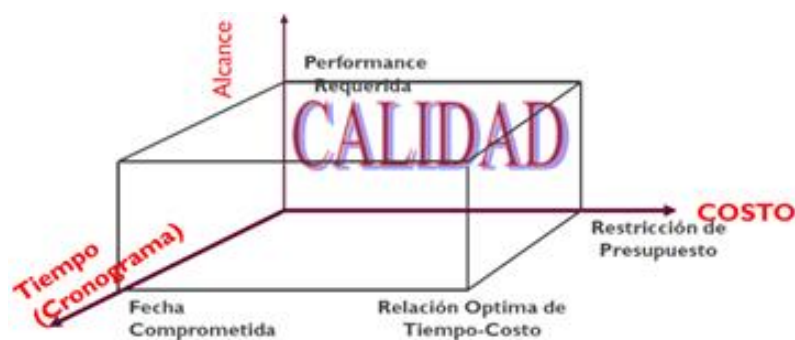
- La administración de proyectos es la organización y coordinación de personas y recursos que permiten desarrollar un producto o servicio de acuerdo con el presupuesto, tiempo y funcionalidades acordadas con el cliente, asegurando que se entregue un producto o servicio de alta calidad.
- La administración de proyectos es una parte fundamental para que el proyecto sea exitoso.
- Tiene 2 grandes actividades: Planificación y Monitoreo
- La administración de proyectos incluye:
 - Identificar los requerimientos.
 - Establecer objetivos claros y alcanzables.
 - Adaptar las especificaciones, planes y el enfoque a los diferentes intereses de los involucrados (stakeholders).

Administración de proyectos dependiendo en ciclo de vida

- La elección de un ciclo de vida afecta de forma directa a la administración que se debe realizar sobre el proyecto, ya que cada uno de los modelos de proceso poseen características y enfoques distintos.
- Por ejemplo, si se realiza un análisis de riesgo y se determina que los requerimientos pueden variar mucho a lo largo del desarrollo, se debería elegir un ciclo de vida en el cual el cambio sea permisible, como los iterativos.
- En este caso, la administración y la planificación del proyecto se ve afectada en la elección del ciclo de vida.

La triple restricción

- Las variables de restricción de un proyecto son 3:
 - Tiempo: Define la fecha de calendario en la cual se compromete a finalizar el proyecto y entregarle el producto resultante al cliente.
 - Costo: Define el costo de todos los recursos implicados en el desarrollo del proyecto. Esto incluye equipamiento, infraestructura, equipos, salarios, entre otros.
 - Alcance: Son los requerimientos que se incluyen en el proyecto y definen la funcionalidad que tendrá el producto a entregar al cliente.
- El balance de estos tres factores afecta directamente la calidad del proyecto, ya que un proyecto de alta calidad es aquel que entrega el producto, servicio o resultado requerido, satisfaciendo los objetivos en el tiempo estipulado y con el presupuesto planificado.
- Es responsabilidad del Líder de proyecto balancear estas variables.
- El concepto de la triple restricción hace referencia a que no es posible fijar de forma arbitraria a las 3 variables, sino que será posible fijar 2 y la tercera se ajustará a lo definido.
- La calidad no se negocia



El desarrollo de Software

- Producto de sw: cada nueva versión es desarrollada incrementalmente en una serie de pasos.

Líder de Proyecto

- En un enfoque tradicional, el líder de proyecto realiza la toma de decisiones en su totalidad sin la inclusión de la opinión del equipo. El líder asigna las tareas que deben realizar los integrantes del equipo, y además, maneja todas las relaciones con todos los stakeholders del proyecto (clientes, gerencias, contratistas, stakeholders en general).
- Usualmente los técnicos no disponen de habilidades para tratar con la gente, por lo que se necesita una figura con aptitudes como:
 - Motivación
 - Organización
 - Innovación
 - Empatía
 - Comunicación
 - Liderazgo
- También cuentan con habilidades duras como: resolución de problemas, identidad administrativa, logro e influencia.
- Las responsabilidades del líder de proyecto son:
 - Definir el alcance del proyecto.
 - Identificar a los involucrados, recursos y presupuesto.
 - Detallar las tareas, estimar los tiempos y requerimientos.
 - Identificar y evaluar riesgos.
 - Preparar planes de contingencia.
 - Controlar hitos y participar en las revisiones de las fases del proyecto.
 - Administrar el proceso de control de cambios.
 - Producir reportes de estado.



Equipo de proyecto

- Es un grupo de personas comprometidas con alcanzar un conjunto de objetivos de los cuales se sienten mutuamente responsables.
- Entre sus características se encuentran:
 - Diversos conocimientos y habilidades (técnicas, de motivación, experiencias, etc.). Deben tener sentimiento de mejora continua y aprendizaje mutuo.
 - Posibilidad de trabajar en equipo
 - Usualmente son un grupo pequeño, lo que mejora la comunicación afectiva.
 - Trabajan en forma de conjunto con espíritu de grupo.
 - Sentido de responsabilidad como una unidad y se enfocan en el cumplimiento de las metas grupales.

Plan de proyecto

- El plan de proyecto, (o también llamado plan de desarrollo de software en el ámbito del software), es un artefacto de gestión que cumple la función de “hoja de ruta” de un proyecto, en la cual se documentan todas las decisiones que se toman a lo largo del desarrollo; en términos de: el proceso, la división de tareas a realizar, asignación de responsabilidades, equipos de trabajo, calendario de desarrollo, actividades de soporte (planes de contingencia, mecanismos para control de cambios, evaluar calidad y valorar riesgos).
- En un plan de proyecto se establece qué se va a hacer (alcance del proyecto), cuándo se va a hacer (calendario), cómo se va a hacer (actividades o tareas para cubrir el alcance) y quién lo va a hacer (responsables de cubrir las actividades).

- Tiene actividades como:
 - Definición del alcance del proyecto
 - Definición de proceso (pasos a cumplir) y ciclo de vida (como cumplimos esos pasos)
 - Estimación
 - Gestión de riesgos
 - Asignación de recursos
 - Programación de proyectos
 - Definición de controles
 - Definición de métricas

Definición de alcance

- Alcance de proyecto: es todo el trabajo y únicamente el trabajo que debe hacerse para entregar el producto o servicio con todas las características y funciones especificadas.
 - Se mide contra el plan de proyecto
- Alcance de producto: son todas las características que pueden incluirse en un producto o servicio
 - Se mide contra la especificación de requerimientos
- Tanto el alcance como el objetivo de un proyecto puede ser modificado a lo largo del proyecto, pero esto no necesariamente tiene que cambiar el alcance del producto. EL alcance de proyecto es menor que el de producto.

Definición de un ciclo de vida

- El ciclo de vida del proyecto es lo que define cuánto de cada tarea se debe hacer y en qué momento del desarrollo del proyecto hacerlo.
- El ciclo de vida de un proyecto define lo siguiente:
 - Qué trabajo técnico debería realizarse en cada fase.
 - Quién debería estar involucrado en cada fase.
 - Cómo controlar y aprobar cada fase.
 - Cómo deben generarse los entregables.
 - Cómo revisar, verificar y validar el producto.
- En metodologías tradicionales, se permite elegir cualquier tipo de ciclo de vida (cascada, iterativo e incremental, espiral, etc.), en cambio en metodologías ágiles esto no es posible (si o si tiene que ser iterativo e incremental).

Estimaciones de software

- En un proyecto de desarrollo de software, se trata de estimar para predecir el valor de un elemento relacionado con el proyecto o con el producto.
- En un enfoque tradicional, esta estimación de valores es realizada únicamente por el líder de proyecto.
- Es tradicional se estima:
 - **Tamaño:** para estimar el tamaño de un proyecto se utilizan los requerimientos tomados en la etapa de requisitos. Esta estimación es una de las más importantes a realizar, ya que el aumento o disminución del tamaño del producto, afecta de gran manera a otros aspectos.
 - **Esfuerzo:** horas lineales que necesito para construir lo definido en el tamaño.
 - **Calendario:** la estimación del calendario del proyecto permite responder al cuándo se van a realizar las tareas definidas.
 - **Costo:** la estimación de costos depende directamente del tamaño y del esfuerzo que se necesite para el desarrollo del software.
 - **Recursos críticos:** cuales son los momentos de tiempo o recursos que me pueden generar un inconveniente.

Gestión de Riesgos

- Un **riesgo** es la probabilidad de ocurrencia de una pérdida o daño que afecte de forma directa al proyecto y pueda comprometer el incumplimiento de su objetivo. Siempre tiene asociada una probabilidad de ocurrencia, y esta puede variar a lo largo del proyecto.
- Los tipos de riesgos que tenemos son:
 - **Riesgo del proyecto:** son aquellos que alteran el calendario o los recursos del proyecto.
 - **Riesgo del producto:** son aquellos que afectan la calidad o el rendimiento del software a desarrollar.
 - **Riesgos organizacionales o de negocio:** son aquellos que afectan a la organización que desarrolla o que adquiere el software.
- Una vez identificados los riesgos se pueden tomar 3 actitudes:
 - **Negación:** hacer de cuenta que no existen
 - **Gestión Reactiva:** esperar a que ocurra y recién ahí ver que hacer
 - **Gestión Proactiva:** ir trabajando sobre el riesgo, generando planes de mitigación (bajan la exposición al riesgo) y de contingencia (permiten actuar cuando el riesgo suceda y poder volver a la normalidad).

Pasos de la Gestión de Riesgos

- 1) **Identificación del riesgo:** se identifican los que suponen mayor amenaza al proceso de ingeniería de SW, al sw a desarrollar o a la organización.
- 2) **Análisis de riesgos:** para cada riesgo identificado se realiza un juicio acerca de la probabilidad y del impacto.
- 3) **Planeación del riesgo:** para cada riesgo analizado, se definen estrategias para manejarlos, que consisten en acciones a tomar para minimizar o evitar el impacto sobre el proyecto como consecuencia de la ocurrencia de este.
- 4) **Monitorización del riesgo y control:** proceso que permite determinar que las suposiciones acerca de los riesgos de producto, proceso y organización no han cambiado.

Métricas

- Se definen para poder medir en valores concretos como vamos con el proyecto. Vamos a dividir el dominio de métricas en 3:
 - **Métricas de proceso:** son las mismas métricas que proyecto, pero despersonalizadas del proyecto, con el foco en saber si el proceso de sw tiene que cambiar porque está mal definido
 - **Métricas de proyecto:** permite ver como voy con el proyecto, para que el líder tome acciones. Tiene una mirada más táctica.
 - Se consolidan para crear métricas de proceso que sean publicas para toda la organización del software.
 - Se clasifican en:
 - Tamaño del producto (apunta a prod)
 - Esfuerzo
 - Calendario (tiempo)
 - Defectos (apunta a prod)
 - Costos
 - **Métricas de producto:** tiene que ver con defectos, miden características del producto final
- Al realizar la planificación del proyecto, es necesario definir de forma clara y no ambigua, cada una de las métricas asociadas a cada dominio, en conjunto con la forma de calcularlas.
- Cada métrica debe representar aumento de información y beneficiar al desarrollo.
- Las métricas las miro depende las características del proyecto, para ver cuales tomo y cada cuanto tiempo.
- Con las métricas voy a realizar monitoreo y control del proyecto, una vez tomadas quiero saber si el proyecto va a cumplir con los objetivos planteados o si debo corregir los desvíos para lograr los objetivos.

- Este monitoreo y control es ir comparando lo planificado con la realidad.

Factores para el éxito de un proyecto

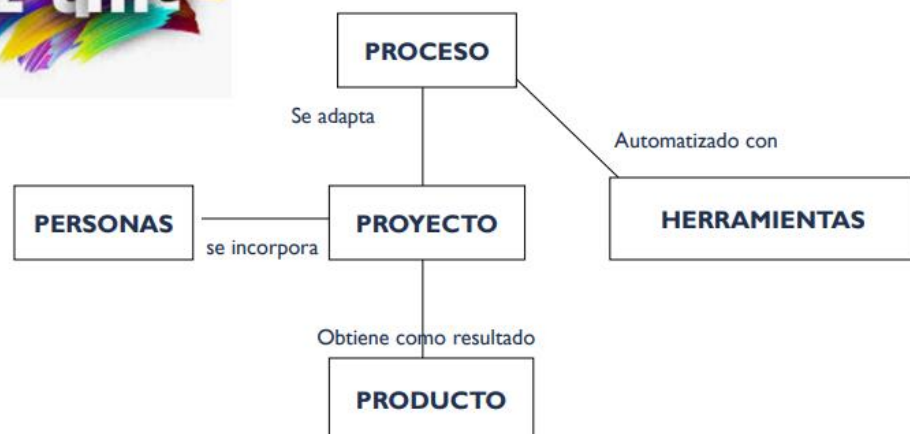
- Monitoreo y feedback
- Tener una misión u objetivo claro
- Comunicación en todos los aspectos

Causas de fracasos de proyectos

- Fallas de toma de REQUERIMIENTOS (el 80%).
- Fallas al definir el problema.
- Planificar basado en datos insuficientes.
- La planificación la hizo el grupo de planificaciones.
- No hay seguimiento del plan de proyecto.
- Plan de proyecto pobre en detalles.
- Planificación de recursos inadecuada.
- Las estimaciones se basaron en “supuestos” sin consultar datos históricos.
- Nadie estaba a cargo.

Vínculo proceso-proyecto-producto en la gestión de un proyecto de desarrollo de software

- Se dice que el proceso, automatizado con herramientas, se adapta al proyecto, al cual se incorporan personas, y de este se obtiene como resultado un producto.
- En el enfoque tradicional, previo a la definición del proyecto, es necesario determinar los objetivos y el ámbito del producto para de esta manera poder realizar las estimaciones pertinentes.
- Los objetivos identifican las metas globales del producto sin especificar como se van a lograr.
- El **proceso** proporciona un marco conceptual en donde se establece un plan completo para el desarrollo del software. Define como se va a desarrollar el software, estableciendo un conjunto de actividades.
- Un **proyecto** de desarrollo está integrado por un equipo de trabajo, dentro del cual deben estar los roles bien definidos, para que cada uno de los integrantes asuma la responsabilidad que le corresponda, y se tengan en claro los alcances de cada uno de estos roles. Realizando actividades para lograr el desarrollo del **producto** deseado
- Las **personas** son la parte más importante, debido a que el desarrollo de software es una actividad humano-intensiva.
- En el desarrollo de un proyecto, se busca utilizar diversas **herramientas** que se adapten y permitan automatizar o facilitar las actividades que están definidas en el proceso adoptado.



No Silver Bullet

- Plantea contrastar progreso del hardware (algo tangible, construible, que se puede mejorar en su construcción), en comparación con el software (intangibile, complejo inherentemente).
- Las balas de plata son aquellas soluciones que podrían hacer frente a la crisis del software: no existe un desarrollo de software que demuestre avances de progresión (simplicidad, fiabilidad y productividad) en la construcción del software en menos de una década.
- **Dificultades que plantea:**
 - **Esenciales:** todo aquello que trae aparejado el construir software: conceptos abstractos, estructuras de datos, algoritmos, invocación de funciones, etc. (carece de precisión y detalles). 4 aspectos la caracterizan:
 - Complejidad:
 - propiedad inherente, debido a la abstracción del software
 - al crecer el tamaño del software, su complejidad crece de manera exponencial, no lineal
 - Esto conduce a muchos problemas: dificultad de comunicación entre miembros de equipos, deficiencias en el producto, errores en cumplimiento de fechas, excesos en costos, etc.
 - Conformidad:
 - el software debe realizar aquello que el cliente necesita, esto es: debe adecuarse a sus requerimientos.
 - Mutabilidad
 - el software está sujeto a constantes cambios. A diferencia de objetos tangibles, el software muta y sufre cambios a lo largo del tiempo. En cambio, en muchos objetos esto no es así, ya que se reemplazan por nuevos productos (no tomo el objeto y lo modifico, creo uno nuevo).
 - Invisibilidad:
 - al ser algo no visualizable, se dificulta mucho trabajar con representaciones geométricas, como puede ser con estructuras mecánicas. Los grafos son una buena herramienta que permiten modelar muchas situaciones de un software.
 - **Accidentales:** tiene que ver con aquellos aspectos que dificultan la construcción del software, pero han sido solucionados a lo largo del tiempo. Esas soluciones son:
 - Lenguajes de programación de alto nivel: permiten abstraer el proceso de construcción del software, independizándose del hardware en el que se ejecutarán. Esto elimina la complejidad propia de una máquina, que nada tiene que ver con el software que se desea construir.
 - Tiempo compartido: reducción de los tiempos de respuesta de un software. Esto permite no desviar el foco de lo que se desea construir.
 - Entornos de programación unificados: facilitan la construcción del software al integrar librerías, formato de archivos, frameworks, etc.

- **Resoluciones de dificultades accidentales**

- Programación orientada a objetos: permite a los diseñadores del software, abstraerse de tecnicismos para expresar su diseño simplemente en un lenguaje de máquina. Esto facilita el diseño, pero sólo elimina una dificultad accidental, no elimina la propia dificultad esencial de diseñar software.
- Inteligencia Artificial: si bien la IA permite resolver muchas cuestiones que reemplazan acciones humanas, siempre tiene que haber un equipo de personas que programe y “enseñe” a esa IA cómo actuar. Con lo cual, el factor humano no se reemplaza del todo.
- Sistemas expertos: son sistemas que aplican la IA y permiten sugerir diferentes situaciones como estrategias de Testing, optimizaciones de código, sugerencias en nombrado de variables, etc. Igualmente, no se quita el factor humano ya que se necesita encontrar expertos y desarrollar técnicas eficientes para extraer lo que ellos saben para destilarlo dentro de bases de reglas de estos sistemas. El prerequisite esencial para construir un sistema experto es tener un experto.
- Programación automática: esto se planteó como un reemplazo a los programadores, pero en lugar de ser eso, terminan siendo un lenguaje de más alto nivel que el programador debe parametrizar.
- Programación gráfica: permite programar a un nivel más alto de abstracción que la programación tradicional, pero sigue estando el factor humano presente, con lo que solo se logra dar una mayor abstracción al programador sin que este tenga la necesidad de aprender sintaxis del lenguaje.
- Verificación de programas: establecer que un programa pasa las pruebas matemáticas de verificación que se proveen. Verifica que las funcionalidades desarrolladas no tengan errores sin verificar que se cumplen todas las necesidades del cliente.
- Environments and tools: La mayoría de los nuevos entornos inteligentes tan solo nos permiten librarnos de errores semánticos y sintácticos simples.
- Estaciones de trabajo poderosas: El incremento de potencia y memoria de las estaciones de trabajo individuales no soluciona el déficit en el desarrollo del software. Una mejora en tiempos de compilación no representa un avance significativo.

- **Resolución de dificultades esenciales**

- Comprar, antes que construir: explorar en el mercado soluciones que se puedan conseguir para reemplazar lo que se quiera construir. Si existe un mercado que lo ha demandado, comprarlo siempre será más barato que construirlo.
- Usar prototipos y refinar requerimientos: la parte fundamental de construir software es saber qué se debe construir. La extracción iterativa de requerimientos y su posterior refinamiento permiten tomar las necesidades de los clientes en sucesivas iteraciones, debido a que ellos mismos no saben expresar ni conocer muchas veces cuáles son esas necesidades que tienen. El desarrollo de prototipos rápidos permite cubrir esa extracción iterativa de requerimientos, al simular interfaces y funciones principales de un software que se necesita construir.
- Desarrollo incremental: Incrementar el software añadiendo funcionalidades mientras estas se prueban, usan y testean. Tener un software con pocas funcionalidades, pero funcionando, es más animador que tener un avance de un software complejo que no funciona.
- Personas capacitadas: Esto permite tener un buen equipo, pero, al fin y al cabo, desarrollar software requiere de creatividad, por lo que priorizar la creatividad en un equipo puede traducirse en grandes resultados

SCRUM

Scrum es un marco ligero que ayuda a las personas, equipos y organizaciones a generar valor a través de soluciones adaptables para problemas complejos. En pocas palabras, Scrum requiere un Scrum Master para fomentar un entorno donde:

- 1) Un propietario del producto (Product Owner) ordena el trabajo de un problema complejo en un Product Backlog.
- 2) El equipo de Scrum convierte una selección del trabajo en un Incremento de valor durante un Sprint.
- 3) El equipo de Scrum y sus partes interesadas (stakeholders) inspeccionan los resultados y realizan los ajustes necesarios para el próximo Sprint.
- 4) Repetir

Scrum es simple. Pruébalo tal cual y determine si su filosofía, teoría y estructura ayudan a alcanzar metas y crear valor. El marco de Scrum es deliberadamente incompleto, solo define las partes necesarias para implementar la teoría de Scrum. Scrum se basa en la inteligencia colectiva de las personas que lo utilizan. En lugar de proporcionar a las personas instrucciones detalladas, las reglas de Scrum guían sus relaciones e interacciones. En el marco se pueden emplear diversos procesos, técnicas y métodos. Scrum envuelve las prácticas existentes o las hace innecesarias. Scrum hace visible la eficacia relativa de la gestión actual, el entorno y las técnicas de trabajo, de modo que se pueden realizar mejoras.

Scrum se basa en el empirismo y el pensamiento Lean. El empirismo afirma que el conocimiento proviene de la experiencia y la toma de decisiones basadas en lo que se observa. El pensamiento Lean reduce los desperdicios y se centra en lo esencial. Scrum emplea un enfoque iterativo e incremental para optimizar la previsibilidad y controlar el riesgo. Scrum involucra a grupos de personas que colectivamente tienen todas las habilidades y experiencia para hacer el trabajo y compartir o adquirir tales habilidades según sea necesario. Scrum combina cuatro eventos formales para la inspección y adaptación dentro de un evento contenedor, el Sprint. Estos eventos funcionan porque implementan los pilares empíricos de Scrum: transparencia, inspección y adaptación.

Pilares de Scrum

- **Transparencia:** El proceso y el trabajo emergentes deben ser visibles para aquellos que realizan el trabajo, así como para los que reciben el trabajo. Con Scrum, las decisiones importantes se basan en el estado percibido de sus tres artefactos formales. Los artefactos que tienen poca transparencia pueden conducir a decisiones que disminuyen el valor y aumentan el riesgo. **La transparencia permite la inspección. La inspección sin transparencia genera engaños y desperdicios.**
- **Inspección:** Los artefactos de Scrum y el progreso hacia objetivos acordados deben ser inspeccionados con frecuencia y diligentemente para detectar varianzas o problemas potencialmente indeseables. Para ayudar con la inspección, Scrum proporciona cadencia en forma de sus cinco eventos. **La inspección permite la adaptación. La inspección sin adaptación se considera inútil. Los eventos de Scrum están diseñados para provocar cambios.**
- **Adaptación:** Si algún aspecto de un proceso se desvía fuera de los límites aceptables o si el producto resultante es inaceptable, el proceso que se está aplicando o los materiales que se producen deben ajustarse. El ajuste debe realizarse lo antes posible para minimizar la desviación adicional. La adaptación se vuelve más difícil cuando las personas involucradas no están empoderadas o no poseen capacidad para autogestionarse. Se espera que un equipo de Scrum se adapte en el momento en que aprenda algo nuevo por medio de la inspección.

Valores de Scrum

El uso exitoso de Scrum depende de que las personas sean más competentes en vivir cinco valores:

Compromiso, Enfoque, Apertura, Respeto y Coraje.

El equipo de Scrum se compromete a lograr sus objetivos y apoyarse mutuamente. Su enfoque principal es el trabajo del Sprint para hacer el mejor progreso posible hacia estos objetivos. El equipo de Scrum y sus partes interesadas están abiertos sobre el trabajo y los desafíos. Los miembros del equipo de Scrum se respetan mutuamente para ser personas capaces e independientes, y son respetados como tales por las personas con las que trabajan. Los miembros del equipo de Scrum tienen el valor de hacer lo correcto y de trabajar en problemas complejos. Estos valores dan dirección al equipo de Scrum con respecto a su trabajo, acciones y comportamiento. Las decisiones que se toman, las medidas tomadas y la forma en que se utiliza Scrum deben reforzar estos valores, no disminuirlos o socavarlos. Los miembros del equipo de Scrum aprenden y exploran los valores mientras trabajan con los eventos y artefactos de Scrum. Cuando estos valores son asimilados por el equipo de Scrum y las personas con las que trabajan, los pilares empíricos de Scrum de transparencia, inspección y adaptación cobran vida construyendo confianza.

El Equipo Scrum (Scrum Team)

La unidad fundamental de Scrum es un pequeño equipo de personas, un equipo Scrum. El equipo Scrum consta de un Scrum Master, un propietario de producto (Product Owner) y desarrolladores. Dentro de un equipo de Scrum, no hay sub-equipos ni jerarquías. Es una unidad cohesionada de profesionales enfocada en un objetivo a la vez, el objetivo del Producto.

Los equipos de Scrum son multifuncionales, lo que significa que los miembros tienen todas las habilidades necesarias para crear valor en cada Sprint. También son autogestionados, lo que significa que internamente deciden quién hace qué, cuándo y cómo. El equipo de Scrum es lo suficientemente pequeño como para permanecer ágil y lo suficientemente grande como para completar un trabajo significativo dentro de un Sprint, por lo general 10 o menos personas. En general, hemos descubierto que los equipos más pequeños se comunican mejor y son más productivos. Si los equipos de Scrum se vuelven demasiado grandes, se debe considerar la posibilidad de reorganizarse en varios equipos Scrum cohesionados, cada uno centrado en el mismo producto. Por lo tanto, deben compartir el mismo objetivo de producto, trabajo pendiente del producto (Product Backlog) y propietario del producto (Product Owner). El equipo Scrum es responsable de todas las actividades relacionadas con los productos, desde la colaboración, verificación, mantenimiento, operación, experimentación, investigación y desarrollo, y cualquier otra cosa que pueda ser necesaria. Están estructurados y empoderados por la organización para gestionar su propio trabajo. Trabajar en Sprints a un ritmo sostenible mejora el enfoque y la consistencia del equipo de Scrum. Todo el equipo de Scrum es responsable de crear un incremento valioso y útil en cada Sprint. Scrum define tres responsabilidades específicas dentro del equipo de Scrum: los desarrolladores, el propietario del producto (Product Owner) y el Scrum Master.

Desarrolladores

Los desarrolladores son las personas del equipo Scrum que se comprometen a crear cualquier aspecto de un Incremento útil (funcional) en cada Sprint. Las habilidades específicas que necesitan los desarrolladores son a menudo amplias y variarán con el dominio del trabajo. Sin embargo, los desarrolladores siempre son responsables de:

- Crear un plan para el Sprint, el Sprint Backlog.
- Inculcar la calidad adhiriéndose a una definición de Hecho.
- Adaptar su plan cada día hacia el Objetivo Sprint.
- Responsabilizarse mutuamente como profesionales.

Propietario del producto (Product Owner)

El Propietario del Producto es responsable de maximizar el valor del producto resultante del trabajo del equipo de Scrum. La forma en que esto se hace puede variar ampliamente entre organizaciones, equipos Scrum e individuos. El Propietario del Producto también es responsable de la gestión eficaz de la pila del producto (Product Backlog), que incluye:

- Desarrollar y comunicar explícitamente el Objetivo del Producto.
- Creación y comunicación clara de elementos de trabajo pendiente del producto.
- Pedido de artículos de trabajo pendiente del producto.
- Asegurarse de que el trabajo pendiente del producto sea transparente, visible y comprendido.

El Propietario del Producto puede hacer el trabajo anterior o puede delegar la responsabilidad a otros. En cualquier caso, el propietario del producto sigue siendo responsable. Para que los Propietarios de Productos tengan éxito, toda la organización debe respetar sus decisiones. Estas decisiones son visibles en el contenido y el orden del trabajo pendiente del producto, y a través del Incremento inspeccionable en la revisión de Sprint. El Propietario del Producto es una persona, no un comité. El Propietario del Producto puede representar las necesidades de muchas partes interesadas en el trabajo pendiente del producto. Aquellos que deseen cambiar el trabajo pendiente del producto pueden hacerlo tratando de negociar con criterio con el Product Owner.

Scrum Master

El Scrum Master es responsable de establecer Scrum tal como se define en la Guía de Scrum. Lo consigue ayudando a todos a comprender la teoría y la práctica de Scrum, tanto dentro del Equipo como en toda la organización. El Scrum Master es responsable de la efectividad del Scrum Team. Lo logra al permitir que el equipo Scrum mejore sus prácticas, dentro del marco de Scrum. Los Scrum Masters son verdaderos líderes que sirven al equipo Scrum y a toda la organización.

El Scrum Master sirve al equipo de Scrum de varias maneras, incluyendo:

- Capacitar a los miembros del equipo en autogestión y multifuncionalidad.
- Ayudar al equipo de Scrum a centrarse en la creación de incrementos de alto valor que cumplan con la definición de hecho.
- Promover la eliminación de los impedimentos para el progreso del equipo Scrum.
- Asegurar de que todos los eventos de Scrum se lleven a cabo, sean positivos, productivos y que se respete el tiempo establecido (time-box) para cada uno de ellos.

El Scrum Master sirve al Propietario del Producto (Product Owner) de varias maneras, incluyendo:

- Ayudar a encontrar técnicas para una definición eficaz de los objetivos del producto y la gestión de los retrasos en el producto.
- Ayudar al equipo de Scrum a comprender la necesidad de elementos de trabajo pendiente de productos claros y concisos.
- Ayudar a establecer la planificación empírica de productos para un entorno complejo.
- Facilitar la colaboración de las partes interesadas según sea solicitado o necesario.

El Scrum Master sirve a la organización de varias maneras, incluyendo:

- Liderar, capacitar y mentorizar a la organización en su adopción de Scrum.
- Planificar y asesorar sobre la implementación de Scrum dentro de la organización.
- Ayudar a las personas y a las partes interesadas a comprender y promulgar un enfoque empírico para el trabajo complejo.
- Eliminar las barreras entre las partes interesadas y los equipos de Scrum.

Eventos de Scrum

El Sprint es un contenedor para todos los eventos. Cada evento en Scrum es una oportunidad formal para inspeccionar y adaptar los artefactos de Scrum. Estos eventos están diseñados específicamente para permitir la transparencia necesaria. Si no se realizan los eventos según lo prescrito, se pierden oportunidades para inspeccionar y adaptarse. Los eventos se utilizan en Scrum para crear regularidad y minimizar la necesidad de reuniones no definidas en Scrum. De manera óptima, todos los eventos se llevan a cabo al mismo tiempo y lugar para reducir la complejidad.

El Sprint

Los Sprints son el latido del corazón de Scrum, donde las ideas se convierten en valor. Son eventos de longitud fija de un mes o menos para crear consistencia. Un nuevo Sprint comienza inmediatamente después de la conclusión del Sprint anterior. Todo el trabajo necesario para alcanzar el objetivo del producto, incluyendo la Planificación (Sprint Planning), Daily Scrums, Revisión del Sprint (Sprint Review) y la Retrospectiva (Sprint Retrospective), ocurren dentro del Sprints. Durante el Sprint:

- No se hacen cambios que pongan en peligro el Objetivo Sprint.
- La calidad no disminuye.
- El trabajo pendiente del producto se refina según sea necesario.
- El alcance se puede clarificar y renegociar con el Propietario del Producto a medida que se aprende más.

Los Sprints permiten la previsibilidad al garantizar la inspección y adaptación del progreso hacia un objetivo del Producto, como mínimo, una vez al mes en el calendario. Cuando el horizonte de un Sprint es demasiado largo, el Objetivo de Sprint puede volverse obsoleto, la complejidad puede aumentar y el riesgo puede aumentar. Los Sprints más cortos se pueden emplear para generar más ciclos de aprendizaje y limitar el riesgo de coste y esfuerzo a un período de tiempo más pequeño. Cada Sprint puede considerarse un proyecto corto.

Existen varias prácticas para pronosticar el progreso, como gráficos de burn-downs, burn-ups, o flujos acumulativos. Si bien han demostrado ser útiles, estos no sustituyen la importancia del empirismo. En entornos complejos, se desconoce lo que sucederá. Solo lo que ya ha sucedido se puede utilizar para la toma de decisiones con vistas a futuro.

Un Sprint podría ser cancelado si el Objetivo del Sprint se vuelve obsoleto. Solo el Propietario del Producto tiene la autoridad para cancelar el Sprint.

1) Planificación de Sprint (Sprint Planning)

El Sprint Planning inicia el Sprint estableciendo el trabajo que se realizará para el mismo. Este plan resultante es creado por el trabajo colaborativo de todo el equipo de Scrum. El propietario del producto (Product Owner) se asegura de que los asistentes estén preparados para discutir los elementos de trabajo pendiente de producto más importantes y cómo se asignan al objetivo del producto. El equipo de Scrum también puede invitar a otras personas a asistir a la planificación del Sprint para proporcionar asesoramiento. La planificación del Sprint aborda los siguientes temas:

- Tema Uno: ¿Por qué este Sprint es valioso?
El Propietario del Producto (Product Owner) propone cómo el producto podría aumentar su valor y utilidad en el Sprint actual. A continuación, todo el equipo de Scrum colabora para definir un objetivo de Sprint que comunique por qué el Sprint es valioso para las partes interesadas. El Objetivo Sprint debe finalizarse antes del final de la Planificación de Sprint.
- Tema dos: ¿Qué se puede hacer este Sprint?
A través del debate con el propietario del producto (Product Owner), los desarrolladores seleccionan los elementos del Product Backlog para incluir en el Sprint actual. El equipo de Scrum puede refinar estos elementos durante este proceso, lo que aumenta la comprensión y confianza. Seleccionar cuánto se puede completar dentro de un Sprint puede ser un desafío. Sin embargo, cuanto más sepan los desarrolladores sobre su rendimiento pasado, su capacidad futura y su definición de hecho, más seguros estarán en sus pronósticos de Sprint.

- Tema Tres: ¿Cómo se realizará el trabajo elegido?
Para cada elemento de trabajo pendiente de producto (Product Backlog item) seleccionado, los desarrolladores planifican el trabajo necesario para crear un incremento que cumpla con la definición de hecho. Esto se hace normalmente mediante la descomposición de elementos de trabajo pendiente (Product Backlog item) del producto en elementos de trabajo más pequeños que se puedan realizar en un día o menos. La forma de hacerlo es según la discreción de los propios desarrolladores. Nadie más les dice cómo convertir los elementos de trabajo pendiente del producto en incrementos de valor.
- El objetivo de Sprint (Sprint Goal), los elementos de trabajo pendiente de producto seleccionados para el Sprint, más el plan para entregarlos se conocen conjuntamente como el trabajo pendiente de Sprint (Sprint Backlog). El Sprint Planning tiene una duración máxima de ocho horas para un Sprint de un mes. Para sprints más cortos, el evento suele ser más corto.

2) **Scrum Diario (Daily Scrum)**

El propósito del Daily Scrum es inspeccionar el progreso hacia el Objetivo Sprint y adaptar el Sprint Backlog según sea necesario, ajustando el próximo trabajo planeado. El Daily Scrum es un evento de 15 minutos (máximo) para los desarrolladores del equipo de Scrum. Para reducir la complejidad, se lleva a cabo al mismo tiempo y lugar todos los días laborables del Sprint. Si el propietario del producto o el Scrum Master están trabajando activamente en los elementos del Trabajo pendiente de Sprint, participan como desarrolladores. Los desarrolladores pueden seleccionar cualquier estructura y técnicas que deseen, siempre y cuando su Scrum diario se centre en el progreso hacia el objetivo de Sprint y produzca un plan accionable para el día siguiente de trabajo. Esto crea enfoque y mejora la autogestión. Los Scrums diarios (Daily Scrum) mejoran la comunicación, identifican impedimentos, promueven una rápida para la toma de decisiones, y en consecuencia, eliminan la necesidad de otras reuniones. El Daily Scrum no es la única vez que los desarrolladores pueden ajustar su plan. Frecuentemente se reúnen durante todo el día para debatir de forma más detalladas sobre la adaptación o replanificación del resto del trabajo del Sprint

3) **Revisión del Sprint (Sprint Review)**

El propósito de la revisión del Sprint es inspeccionar el resultado del Sprint y determinar futuras adaptaciones. El equipo de Scrum presenta los resultados de su trabajo a las partes interesadas clave y se discute el progreso hacia el Objetivo de Producto. Durante el evento, el equipo de Scrum y las partes interesadas revisan lo que se logró en el Sprint y lo que ha cambiado en su entorno. En base a esta información, los asistentes colaboran en qué hacer a continuación. El trabajo pendiente del producto también se puede ajustar para satisfacer nuevas oportunidades. Sprint Review es una sesión de trabajo y el equipo de Scrum debe evitar limitarla a que se convierta en una simple presentación. La revisión de Sprint es el penúltimo evento del Sprint y se utiliza en un plazo máximo de cuatro horas para un Sprint de un mes. Para sprints más cortos, el evento suele ser más corto.

4) **La retrospectiva del Sprint (Sprint Retrospective)**

El propósito de la retrospectiva Sprint es planificar formas de aumentar la calidad y la eficacia. El equipo de Scrum inspecciona cómo fue el último Sprint con respecto a individuos, interacciones, procesos, herramientas y su definición de Hecho. Los elementos inspeccionados a menudo varían según el dominio del trabajo. Las suposiciones que los desviaron se identifican y se exploran sus orígenes. El equipo de Scrum analiza qué fue bien durante el Sprint, qué problemas encontró y cómo esos problemas fueron (o no fueron) resueltos. El equipo de Scrum identifica los cambios más útiles para mejorar su eficacia. Las mejoras más impactantes se abordan lo antes posible. Incluso se pueden agregar al Sprint Backlog para el próximo Sprint. La retrospectiva Sprint concluye el Sprint. Se utiliza un intervalo de tiempo de hasta un máximo de tres horas para un Sprint de un mes. Para sprints más cortos, el evento suele ser más corto.

5) **Refinamiento del product Backlog**

Artefactos de Scrum

Los artefactos de Scrum representan trabajo o valor. Están diseñados para maximizar la transparencia de la información clave. Por lo tanto, cada uno de los que los inspecciona tienen la misma base para la adaptación. Cada artefacto contiene un compromiso para garantizar que proporciona información que mejora la transparencia y el enfoque con el que se puede medir el progreso:

- Para el trabajo pendiente del producto es el objetivo del producto.
- Para el Sprint Backlog es el Sprint Goal.
- Para el Incremento es la Definición de Hecho.

Estos compromisos existen para reforzar el empirismo y los valores de Scrum para el equipo de Scrum y sus partes interesadas.

1) Pila del producto (Product Backlog)

El trabajo pendiente del producto es una lista emergente y ordenada de lo que se necesita para mejorar el producto. Es la única fuente de trabajo emprendida por el equipo Scrum. Los elementos de trabajo pendiente de producto que puede ser hecho por el equipo de Scrum dentro de un Sprint se consideran listos para su selección en un evento de planificación de Sprint. Por lo general adquieren este grado de transparencia después de las actividades de refinación. El refinamiento de Backlog del producto es el acto de descomponer y definir aún más los elementos de trabajo pendiente del producto en artículos más pequeños y precisos. Esta es una actividad en curso para agregar detalles, como una descripción, un pedido y un tamaño. Los atributos a menudo varían con el dominio del trabajo. Los desarrolladores que realizarán el trabajo son responsables del tamaño. El Propietario del Producto (Product Owner) puede influir en los desarrolladores ayudándoles a entender y seleccionar mejores alternativas.

Compromiso: Objetivo del producto (Product Goal)

El objetivo del producto (Product Goal) describe un estado futuro del producto que puede servir como objetivo para el equipo Scrum contra el cual planificar. El objetivo del producto se encuentra en el trabajo pendiente del producto (Product Backlog). El resto del trabajo pendiente del producto surge para definir "qué" cumplirá el objetivo del producto. Un producto es un vehículo para entregar valor. Tiene un límite claro, partes interesadas conocidas, usuarios o clientes bien definidos. Un producto podría ser un servicio, un producto físico o algo más abstracto. El objetivo del producto es el objetivo a largo plazo para el equipo Scrum. Deben cumplir (o abandonar) un objetivo antes de asumir el siguiente.

2) La pila del Sprint (Sprint Backlog)

El Trabajo pendiente de Sprint se compone del objetivo sprint (por qué), el conjunto de elementos de trabajo pendiente de producto seleccionados para el Sprint (qué), así como un plan accionable para entregar el incremento (cómo). El Trabajo pendiente de Sprint es un plan por y para los desarrolladores. Es una imagen muy visible y en tiempo real del trabajo que los desarrolladores planean realizar durante el Sprint para lograr el Objetivo Sprint. Por lo tanto, el Sprint Backlog se actualiza a lo largo del Sprint a medida que se aprende más. Debe tener suficientes detalles para que puedan inspeccionar su progreso en el Scrum Diario.

Compromiso: Sprint Goal

El Sprint Goal es el único objetivo para el Sprint. Aunque el objetivo de Sprint es un compromiso de los desarrolladores, proporciona flexibilidad en términos del trabajo exacto necesario para lograrlo. El Objetivo Sprint también crea coherencia y enfoque, animando al equipo de Scrum a trabajar juntos en lugar de en iniciativas separadas. El objetivo de Sprint se crea durante el evento Sprint Planning y, a continuación, se agrega al Trabajo pendiente de Sprint. A medida que los desarrolladores trabajan durante el Sprint, tienen en cuenta el objetivo de Sprint. Si el trabajo resulta ser diferente de lo que esperaban, colaboran con el propietario del producto para negociar el alcance del Trabajo pendiente de Sprint dentro del Sprint sin afectar al objetivo de Sprint.

3) Incremento (Increment)

Un Incremento es un paso de hormigón hacia el Objetivo del Producto. Cada Incremento es aditivo a todos los Incrementos anteriores y verificado a fondo, asegurando que todos los Incrementos funcionen juntos. Para proporcionar el valor, el incremento debe ser utilizable. Se pueden crear varios incrementos dentro de un Sprint. La suma de los Incrementos se presenta en la Revisión Sprint apoyando así el empirismo. Sin embargo, un Incremento puede ser entregado a las partes interesadas antes del final del Sprint. La revisión de Sprint nunca debe considerarse una puerta para liberar valor. El trabajo no se puede considerar parte de un Incremento a menos que cumpla con la Definición de Hecho.

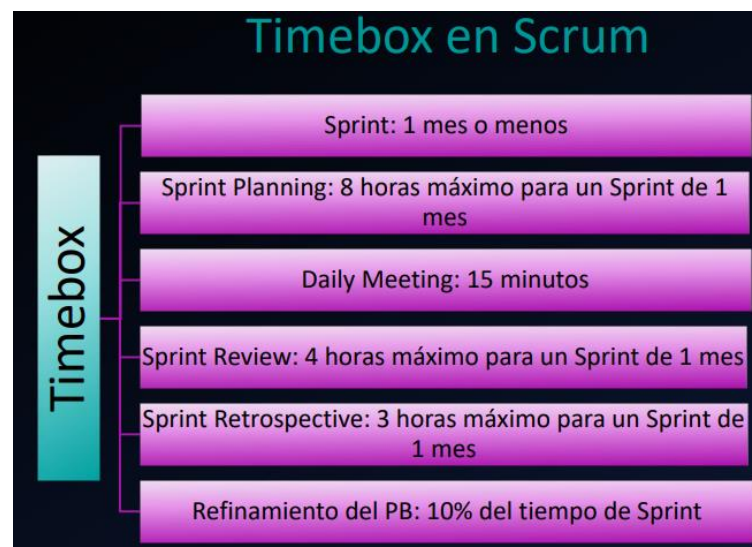
Compromiso: Definición de Hecho (Definition of Done)

La Definición de Hecho es una descripción formal del estado del Incremento cuando cumple con las medidas de calidad requeridas para el producto. En el momento en que un elemento de trabajo pendiente de producto cumple con la definición de hecho, se crea un incremento. La definición de Hecho crea transparencia al proporcionar a todos una comprensión compartida de qué trabajo se completó como parte del Incremento. Si un elemento de trabajo pendiente de producto no cumple con la definición de hecho, no se puede liberar, ni siquiera presentar en la revisión de Sprint. En su lugar, vuelve al Trabajo pendiente del producto para su consideración futura.

Si la definición de hecho para un incremento forma parte de los estándares de la organización, todos los equipos de Scrum deben seguirla como mínimo. Si no es un estándar organizativo, el equipo de Scrum debe crear una definición de hecho adecuada para el producto. Los desarrolladores deben ajustarse a la definición de Hecho. Si hay varios equipos de Scrum trabajando juntos en un producto, deben definir y cumplir mutuamente con la misma definición de hecho.

Timebox en Scrum

- Esto existe para que no se pierda más tiempo del necesario en las distintas tareas y para aprender a negociar en términos del alcance del producto y no del tiempo o los costos (relacionado con la triple restricción).
- Si no se llega al final de un sprint, en vez de extender el tiempo, se entrega menos. De esta manera, nos volvemos más confiables, con un ritmo de trabajo sostenible.
- Todas las ceremonias son Timebox. El refinamiento del Product Backlog al ser una actividad continua, no tiene definido un tiempo exacto, sino que el tiempo se expresa en términos porcentuales sobre la definición del Sprint.



Definición de Listo (DoR)

- El DoR es un criterio que define cuando una US está lo suficientemente bien formulada como para poderse incluir en un Sprint. Por ejemplo, que cierta US debe tener un prototipo, entonces cuando el prototipo esté realizado, el DoR dirá “se ha definido el prototipo para la US”.
- El DoD es un criterio que dice cuando una historia está lo suficientemente bien implementada como para entrar al Sprint Review y ser mostrada al PO, entonces él será el encargado de aprobar o no la US y en caso de que esté aprobada decidirá si desea ponerla en producción.
- En el DoD se arma un checklist con todo lo que debe cumplir una US para entrar en producción.

Definición de "Listo" (Ready)		Definición de Hecho (DONE)	
☐	Valor de negocio claramente expresada.	☐	Diseño revisado
☐	Detalles suficientemente comprendidos por el Equipo de forma tal que puedan tomar una decisión informada sobre si pueden completar el ítem del product Backlog (PBI).	☐	Código Completo
☐	Dependencias identificadas y no hay dependencias externas que puedan impedir que el PBI se complete.	☐	Código refactorizado
☐	El equipo ha sido asignado adecuadamente para completar el PBI.	☐	Código con formato estándar
☐	El PBI ha sido debidamente estimado y es lo suficientemente pequeño para ser completado en un Sprint.	☐	Código Comentado
☐	Los criterios de aceptación son claros y testeables.	☐	Código en el repositorio
☐	Los criterios de performance si hay, son claros y testeables.	☐	Código Inspeccionado
☐	El equipo comprende como mostrar el PBI en la Sprint Review.	☐	Documentación de Usuario actualizada
		☐	Probado
		☐	Prueba de unidad hecha
		☐	Prueba de integración hecha
		☐	Prueba de sistema hecha
		☐	Cero defectos conocidos
		☐	Prueba de Aceptación realizada
		☐	En los servidores de producción

Capacidad del equipo en un Sprint

- La capacidad es una métrica que utiliza SCRUM para determinar cuánta cantidad de trabajo puede comprometer un equipo para un determinado Sprint.
- La capacidad se estima y se mide.
- Una de las cosas que se hace en el Sprint Planning es determinar la capacidad del equipo, y entonces, teniendo en cuenta la capacidad, se contrasta en cuántas US del Product Backlog se pueden incorporar en el Sprint Backlog.
- Para equipos más maduros, la capacidad puede ser estimada en Story Points y para equipos menos maduros se puede estimar en horas ideales.



Ambiente de trabajo

- Abierto: el silencio absoluto es un mal signo
- Pizarrones
- El Equipo define sus horarios

Herramientas de Scrum

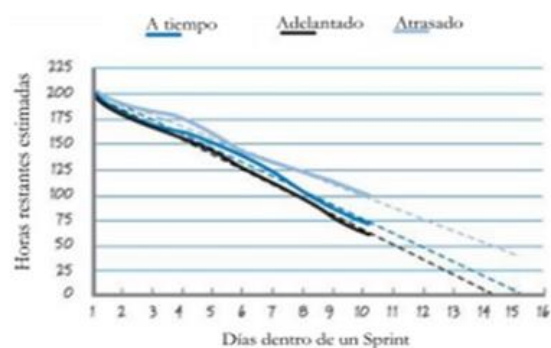
- **Tarjeta de Tarea:** es una tarjeta que consta de 5 partes
 - 1. Código del Ítem Backlog: es un identificador del ítem en el Product Backlog.
 - 2. Nombre de la actividad: suele ser una frase verbal que define qué es lo que hay que hacer.
 - 3. Iniciales del responsable de realizar la tarea.
 - 4. Número de día del sprint.
 - 5. Estimación del tiempo para finalizar la tarea.
- **Tablero:** Tablero utilizado por el Equipo de Desarrollo en el cual se colocan las tarjetas de tareas a realizar durante el sprint. Se encuentra dividido en 5 secciones:
 - 1. Story: el nombre de la historia.
 - 2. To Do: aquí se encuentran las tareas por hacer en el Sprint.
 - 3. Work In Process: aquí se encuentran las tareas que se están realizando.
 - 4. To Verify: aquí se encuentran las tareas finalizadas que deben verificarse.
 - 5. Done: aquí se encuentran las tareas terminadas: finalizadas y verificadas.
- Lo ideal es que el nivel de granularidad de las tareas sea lo más fino posible, de tal forma que se detalle profundamente el avance del sprint, teniendo varias tareas en cada sección del tablero.

Gráficos del Backlog

Son gráficos que brindan información acerca del progreso de un Sprint, de una release o del producto. El backlog de trabajo es la cantidad de trabajo que queda por ser realizado (en horas), mientras que la tendencia del backlog compara esta cantidad con el tiempo (medido en días).

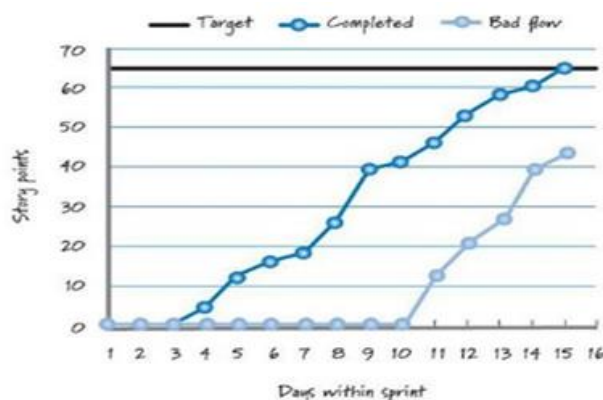
- **Sprint Burn-Down Chart**

- Visualiza con una curva descendente cuánto trabajo queda para terminar. En el eje de las abscisas se coloca el tiempo que va desde el inicio del sprint. En el eje de las ordenadas se coloca los puntos de historia a realizar en el sprint.



- **Sprint Burn-Up Chart**

- Visualiza con una curva ascendente cuánto trabajo se ha completado a lo largo de la release, es decir que visualiza los puntos de historia terminados en cada sprint.



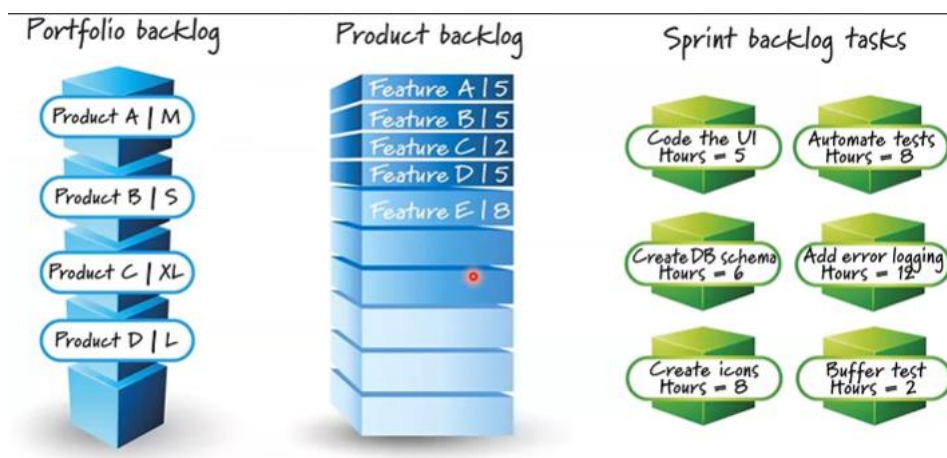
- **Combined Burn Chart**

- Visualiza cuánto trabajo queda para terminar y cuanto trabajo se ha realizado.

Niveles de Planificación

Nivel	Horizonte	Quién	Foco	Entregable
Portfolio	1 año o más	Stakeholders y Product Owners	Administración de un Portfolio de Producto	Backlog de Portfolio
Producto	Arriba de varios meses o más	Product Owner y Stakeholder	Visión y evolución del producto a través del tiempo	Visión de Producto, roadmap y características
Release	3 (o menos) a 9 meses	Equipo Scrum entero y Stakeholders	Balancear continuamente el valor del cliente y la calidad global con las restricciones de alcance, cronograma y presupuesto	Plan de reléase
Iteración	Cada iteración (de 1 semana a 1 mes)	Equipo Scrum entero	Que aspectos entregar en el siguiente Sprint	Objetivo del Sprint y Sprint Backlog
Día	Diaria	Scrum Master y Equipo de desarrollo	Cómo completar lo comprometido	Inspección del progreso

Que y cuando estimar



- Las tareas se estiman en horas ideales y las US en stories point.

Métricas en ambientes ágiles

- **Capacidad**

- Es una métrica de proyecto que mide el compromiso de un equipo para un determinado sprint, en horas de trabajo ideales. Es por esto que, se utiliza para planificar, ya que permite definir cuántas historias de usuario se van a tomar del Product Backlog para implementar en el próximo sprint.
- Se estima al principio de un sprint, por lo que también se la llama velocidad estimada. Se puede estimar en horas ideales o en Story Points.

- **Velocidad**

- Velocidad/ Velocity es una medida (métrica) del progreso de un equipo. Se calcula sumando el número de story points (asignados a cada user story) que el equipo completa durante la iteración .
- Se cuentan los story points de las users Stories que están completas, no parcialmente completas .
- La Velocidad corrige los errores de estimación.